A Hybrid Rasterized Ray-Traced 3D Graphics Renderer

Toby Hothersall

B.Sc. (Hons) Computer Science

Declaration

I certify that the material contained in this dissertation is my own work and does not contain unreferenced or unacknowledged material. I also warrant that the above statement applies to the implementation of the project and all associated documentation. Regarding the electronically submitted work, I consent to this being stored electronically and copied for assessment purposes, including the School's use of plagiarism detection systems in order to check the integrity of assessed work. I agree to my dissertation being placed in the public domain, with my name explicitly included as the author of the work.

Name: Toby Hothersall
Date: 15/04/2026

Abstract

This project aims to develop a 3D rendering engine with separate rasterization and ray tracing pipelines. It aims to do this on the CPU, with a single thread, and implement the rendering equation and bidirectional distribution functions, in order to achieve physically based rendering.

It explains in great detail the rasterization and ray tracing pipelines, acceleration data structures, programmable shading, as well as the linear algebra involved with 3D graphics. It shows class diagrams and code execution flow diagrams for the design of the system; the algorithms implemented; and data related to the speed of each pipeline, while rendering a varying number of triangles at varying resolutions.

The project shows the viability of rasterization and ray tracing, and at what scene complexity *interactive real time rendering*, *real time rendering*, and *offline rendering* is possible for both pipelines. It shows that modern CPUs are more capable of 3D graphics than is typically shown, and is not restricted to the GPU. In a lot of cases, GPUs are not needed to achieve desired 3D rendering baselines, and the use case of 3D rendering can even be extended to embedded systems where there is very tight performance requirements, and limited hardware.

Acknowledgments

I'd like to thank my project supervisor Andrew Scott, for giving valuable help and suggestions during the duration of the project.

Contents

Chapter 1

Introduction

1.1 Overview

Computer graphics has grown into quite a broad field in modern times, influenced by the development of CAD software, CGI in movies, television and animation, and the game industry. The field is generally split into 3 categories: ***offline rendering***, where rendering is not restricted to a time restraint, and individual frames often take upwards of days to render; ***real-time rendering***, where scenes are rendered as they are viewed; and ***interactive real-time rendering***, where scenes are rendered as they are viewed, and the user can interact with it causing those scenes to change in real time.

Since offline rendering doesn't have any time constraints, the quality, realism, and variety of rendering techniques available is broader than in real-time or interactive real-time rendering. This is why offline rendering has seen its most use in the film industry, especially with 3D animation where techniques like path tracing, photon mapping, and instant global illumination have been used to achieve physically correct and/or photo-realistic scenes.

Applications working in real-time, however, can't afford these time consuming algorithms and techniques, and have been predominantly using the triangle rasterization pipeline as its primary rendering technique. So far, this has worked well to simulate all the effects needed, and the increasing performance of the GPU has allowed this highly parallelisable algorithm to be used effectively. Recently the performance of the GPU has increased so much that access to some of the more complex offline rendering techniques has been becoming available for real-time rendering. Specifically, ray tracing has been seeing use in real-time rendering as an approximation to the more computationally intensive path tracing.

In the early days of computing, the CPU was the only processor available for use in graphics programming, but it was realized that the highly parallel nature of graphics and its matrix operations could be utilized by a processor with simpler cores, but a lot of them. The GPU is this processor, and has become a highly sophisticated, and powerful device. Each individual core is less powerful than a CPU core, however, and is limited to a smaller instruction set.

1.2 Problem Definition and Aims of the Project

Even though the GPU is objectively better for graphics programming, the CPU has also become more powerful over the years, and is still a very capable processor. Most renderers take for granted access to GPUs, and there is little to no research on how rendering techniques perform on modern CPUs. There is no research into how rasterization or ray tracing performs on CPUs. This didn't use to be the case, since before GPUs were invented, all rendering was done on CPUs, and the CPUs of today are clearly much faster than old CPUs.

The overall aim of this project is to develop an engine that runs on the CPU and is capable of both rasterization and ray tracing, for 3D scenes. It will contain dynamic scenes that update in real time, where objects either run on the ray tracing pipeline, or rasterization pipeline. Objects running on the ray tracing pipeline will have access to programmable shaders and BSDFs.

The goals of this project are:

- Implement a hybrid rasterized ray traced 3D graphics renderer on the CPU.
- For the raster pipeline and ray tracing pipeline:
  - See how the rendering speed changes with triangle count and resolution
  - Determine the maximum complexity that a scene can be but still achieve real-time frame rates.
  - Determine the maximum complexity that a scene can be but still achieve interactive real-time frame rates.
  - Determine the maximum complexity that a scene can be but still achieve offline rendering frame rates.
- Give a qualitative comparison of the rasterization and ray tracing pipelines, determining their use cases.

1.3 Thesis Overview

This thesis is split into four main chapters: background, design, implementation, and testing and evaluation, together with the introduction and conclusion chapters.

The background chapter will bring you up to date on the theory required to understand this thesis including rasterization, ray tracing, acceleration data structures and the rendering equation. It will also describe some other work done in this field and how they relate to this thesis.

The design chapter is more detailed than a high-level view and will explain how the rendering engine works from a specification level view. It will explain the architecture of the code, as well as how the data is stored, and the main execution flow for a single draw call.

The implementation chapter will go beyond this and give fine-grained details of how the engine works, and how some areas of it changed over time. It will explain the mathematics related to 3D rendering, as well as the main rendering pipeline and shading interface.

The testing and evaluation chapter will test the performance of the renderer with varying levels of complex scenes, and evaluate the rasterizer and the ray tracer, comparing them against one another, in both a qualitative and quantitative manner.

Finally, the conclusions and future work chapter will give an overall conclusion to the project, providing suggested revisions, lessons learned, and areas for future work.

Chapter 2

Background

This chapter will provide the necessary information needed to understand the thesis, including an overview of the two rendering pipelines being implemented, acceleration data structures, the rendering equation and programmable shaders. It will also look into previous work done in 3D rendering and how it relates to this project.

2.1 The Rasterization Pipeline

The purpose of the rasterization rendering pipeline is to render a 2D image given a set of triangle primitives, and, optionally, a virtual camera, light sources, and shaders. It is split into four main stages in the following order: the ***application stage***, the ***geometry processing stage***, the ***rasterization stage***, and the ***fragment processing stage***. Each stage takes as input the output of the previous stage, and gives an output for the next stage. There is a distinction between the logical pipeline and the physical one that is usually implemented; this section describes the logical pipeline[16].

The first stage is the application stage. This stage is for an application using rendering and graphics to generate what it wants to render. It could be a diagram produced in CAD software, a frame for a video game or animated film, for example. Regardless, it should output a list of geometric primitives[1] for the next stage to process. Primitives are usually either a list of triangles, or a list of quadrilaterals (quads), or a combination of the two; other primitives exist but not all renders have functionality to support them. Each triangle or quad will be made up of vertices, and it is the application stage's job to determine properties about each vertex that will be used in later sections of the pipeline.

The job of the geometry processing stage is to conduct any per-triangle or per-vertex shading, and to project these triangles to the screen. Usually, all the primitives inputted to this stage are split into different objects/meshes, that are distinct and don't share any data. Each of these objects will have a set of triangle data, along with a transformation matrix. A virtual camera, along with its transformation matrices, will also be given. It is split into the mandatory vertex shading stage, as well as optional tessellation, geometry shading, and clipping stages.

The vertex shading stage will transform all objects from object space to world space, from world space to camera space, from camera space to clip space, and finally from clip space to screen space. The transformation from camera space to clip space is also where

[1] Smallest piece of geometric data that makes up a mesh

projection occurs; the transformation matrix will project all that is to be rendered into a unit cube known as the canonical view volume, and all coordinates in this space are known as normalised device coordinates. Technically it can just transform objects straight from object space to screen space, but for most scenes, having the functionality to separately change the position of each object, the camera, and the screen space being rendered to, makes splitting it up into separate stages with separate transformations worth while. It will then output a single set of triangles, that have been transformed to screen space, to the rasterization stage.

The optional tessellation stage is used to generate more triangles/quads. If a mesh is given with 100 polygons (triangles or quads), the tessellation stage could be used to recursively split these into more and more polygons. Triangulation is used to split polygons into triangles, while tessellation is used to just create more polygons, regardless of the type. This is used to add more detail to meshes so that shading taking place in later stages in more detailed.

The geometry shading stage is a programmable stage, that the application programmer can use to add more effects to the rendered scenes. It can for example, duplicate and offset triangles, and perform transformations on them. The clipping stage is used to 'clip' away unused triangles that are outside of the rendered screen, and to cut of parts of triangles that are on the border; it is used to make the subsequent stages more efficient by not wasting resources on triangles that will never make it to the screen.

Regardless on what optional stages are used, the geometry processing stage will only output triangles, not quads, or any other type of polygon. Some implementations only allow triangles to be inputted, and will not have a triangulation stage, whereas others allow quads and other polygons to be inputted, and thus require a triangulation stage.

The rasterization stage is rather simple. Its job is to calculate what pixels each triangle covers, and to also handle visibility determination[2]. Visibility determination is usually solved by the Z buffer, this is a buffer that stores the location of any rendered pixel in the z axis, and is indexed by its x, and y coordinates (the z axis goes into the screen). When a new pixel is rasterized, it is checked against the Z buffer, and if it is closer to the camera, it replaces the pixel that is already there. There are various algorithms to rasterize different polygons, but most renderers only provide functionality to rasterize triangles.

The last stage in the pipeline is the fragment-processing (or pixel-processing) stage. To understand this stage, you need to know the distinction between a 'fragment' and a 'pixel'. A pixel is an atomic piece of colour on your screen that a rendering application can render to. A fragment can be thought of as a 'virtual pixel' or a 'pixel-to-be', because it does not have a physical location on the screen, but from the renderers perspective it is the screen. The renderer will render into an array of fragments that represent the screen, but the final location of this screen on your computer is determined by the program using the renderer. For example, a window manager[3] could allow a program to render into a 480 by 320 screen, but once the array of fragments is calculated, the window manager itself will map this to a specific location on the screen.

The job of the pixel-processing stage is to one: do fragment-shading, where the final

[2]Different primitives can cover the same pixel on the screen. Visibility Determination works out which is closest to the screen and thus which one should be rendered

[3]A window manager is a piece of software that manages the creation, deletion, and lifetime of windows

colour of a fragment is determined by what is known as a shader; and two: do screen-mapping, where what is rendered is mapped to the correct location in the viewport[4].

2.2 Ray Tracing

Ray tracing is an alternate method of rendering a scene than rasterization. It takes in the same basic primitives, and outputs an image in the same 2D form, but generates this image in a completely different way. The term 'ray tracing' is actually quite broad and is an umbrella term for several different rendering techniques, including ray casting, recursive ray tracing, and path tracing, with 'ray tracing' being used colloquially for recursive ray tracing. Regardless, ray tracing occurs when all meshes have been transformed to camera space[5].

Ray casting is the most basic form of ray tracing and consists of shooting rays into a scene, one ray per pixel, to determine the colour of each pixel. For each ray, the closest triangle in the scene is determined, and the intersection point with said triangle is determined; the colour of this triangle is then used as the colour of the ray, and hence the colour of the pixel it represents.

Recursive ray tracing is an extension of this where each ray-triangle intersection generates more rays... and those rays will generate more rays... and on recursively. Each time a ray intersects with a primitive, that primitive's shader program will be called, and within this shader, secondary rays will be generated in different directions. This allows secondary effects like reflection, refraction, and shadows to be rendered. This is obviously more computationally expensive than basic ray casting, but yields better quality results.

Path tracing is the most complex version of ray tracing. It is used to simulate global illumination[5] and achieve photorealistic results. It does this by sampling multiple primary rays per pixel, with each ray randomly bouncing around the scene following a probability distribution. It then averages all these samples to determine the final colour of each pixel. This is obviously the most computationally expensive form of ray tracing, but it allows simulation of more complex effects like caustics and soft shadows[19].

There are three stages to the general ray tracing algorithm: ray generation, ray-scene intersection, and ray colour determination. Ray generation refers to the generation of primary rays, by the camera, for each pixel. Then for each ray, intersection checks are done to determine the triangle which the ray intersects. There are many different accelerated data structures that can be used to do this, including kd-trees, binary space partitioning, and bounding volume hierarchies. The last bit is colour determination/resolution, where a 'shader' is called for the primitive that is hit, and this shader calculates the colour of the pixel. This is where secondary ray generation is done for path tracing and recursive ray tracing.

[4]The viewport is what the renderer sees as the canonical screen, even if it is just a buffer that another program uses

[5]Global illumination is where instead of having multiple lights to create a scene, there is a single light: the sun, that handles all illumination

2.3 Acceleration Data Structures

Obviously, producing a ray for each pixel, then checking them each against all triangles in a scene to determine whether or not they intersect, yields abysmal performance. For a solution that generates one ray per pixel, it is O(w * h * t) per pixel, where w is the width of the screen, h the height of the screen, and t is the number of triangles in the scene.

A common solution to this is to use acceleration data structures. Acceleration Data Structures (ADS) are data structures designed to decide as quickly as possible which primitives in a scene a ray intersects, and by extension which primitive it intersects first, by rejecting all primitives that we know for certain the ray doesn't intersect.

The most common accelerated data structure used today are: Bounding Volume Hierarchies (BVHs), kd-trees, and Quadtrees/Octrees[1].

There are three different metrics used to measure how good an ADS is: memory consumption, building time, and traversal time. Typically, building time and traversal time are inversely proportional, because the more time you spend building the data structure, the better it will be. For static scenes, or scenes that change very rarely, the building time isn't an issue, and you'd typically prioritise reducing the traversal time as much as possible. In the case of dynamic scenes that change frequently, you'd what a balance of building time and traversal time that is dependent on how frequently the scene changes; there are also techniques to modify an ADS as the scene changes, rather than rebuild it completely.

There is also a point of how you use the data structures. They can be used on a per mesh basis, where a data structure is built for each mesh independently, meaning that each ray looks through each mesh linearly, while looking through the triangles of each mesh with the time complexity of the chosen data structure. This has the advantage of not having to combine the triangles of all meshes into a single buffer before generation, reducing building time, but does increase traversal time since the triangles will be unevenly spread across each mesh. The alternative is to use them on a per scene basis, where the entire scene has one ADS, and you do combine the triangles of each mesh into a single buffer before generating the ADS. This has the advantage of a faster traversal time, but does increase building time.

Bounding Volume Hierarchies are an object-partitioning data structure stored as a binary tree of bounding volumes, where each node's bounding volume is a subset of its parents bounding volume, and the union of two sister nodes equals the bounding volume of their parent node. Each leaf node references a set of primitives, and each inner node references two child nodes. Starting at the root node, as you go down the tree, the set of possible leaf nodes that intersect the ray halves, until the leaf node that contains the correct primitive is found.

While you can vary the type of primitive stored in one BVH, and vary the type of bounding volume over primitives, it is standard to store one type of primitive (typically triangles), and use one type of bounding volume for the whole data structure. The two most common bounding volumes are spheres, and axis aligned bounding boxes (aabbs). Aabbs are defined by a start coordinate, and end coordinate, and are simply cubes, that are aligned to the axis of the coordinate space used after objects are transformed to camera space. Spheres and aabbs are used because of their fast and simple ray intersection

algorithms[15].

The grid method, is a space-partitioning method, where the coordinate space is split into a large grid of aabbs, where the size of each aabb is predetermined. Then, when a ray intersects the scene, you loop through each primitive inside the first grid cell that the ray intersects, then the next grid cell, and so on, until a maximum number of cells have been searched, or a primitive is found. This has the disadvantage that primitives are not evenly distributed among the cells of the grid, and one cell could contain a large number of primitives. To circumvent this, increasing the resolution of the grid (decreasing the size of each cell) will reduce the number of primitives in each gird, but this results in a large number of empty cells and is not very memory efficient.

Octrees are an extension of this, where the resolution of the grid only increases if there are many primitives inside it, in order to balance the grid. They are stored as 8-ary trees where each cell references a set of primitives, and has up to 8 child nodes. The root node stores as child nodes the 8 octants of the 3D coordinate space up to the boundary of the scene, and recursively splits each octant into 8 more as you traverse down the tree. Splits only occur if a node contains more than a certain number of primitives, and only split up to a certain number of maximum splits.

One complication of using grids or octrees is that primitives may overlap multiple octants. If this is the case, the primitive must either be stored in all overlapping nodes, or be split into multiple distinct primitives.

A Binary Space Partition Tree (BSP) is a space-partitioning data structure that recursively splits the coordinate space in two via arbitrary hyperplanes. A kd-tree is a specialisation of this where every hyperplane must be perpendicular to a base axis. kd-trees share the same complication of overlapping primitives as octrees do. The split at any given point in the construction of a kd-tree is found in the same way as for bounding volume hierarchies, using criteria such as the spacial median, or surface area heuristic[18].

BVHs and kd-trees are both stored as binary trees, and thus the number of levels of the tree is O(log n), and if an efficient splitting algorithm of O(n) is used, then the complexity of tree construction is O(n log n). Octrees also have a construction time complexity of O(n log n). BVHs, kd-trees, and octrees all have a traversal time complexity of O(log n).

2.4 Programmable Shaders

Programmable shading refers to the way that the colour of any ray can be computed in an arbitrary way by shader programs, so long as they follow a specification in terms of the inputs and output of the function - it doesn't even have to use all the inputs.

There are many different kinds of shaders including: camera shaders, responsible for ray generation and casting primary rays; surface shaders, responsible for computing the colour of a ray once it intersects a primitive, including the ability to shoot secondary rays, and shadow rays; and environment shaders, responsible for when rays miss all geometric primitives, simulating effects like skylight.

The most important of these is surface shaders, and while their implementation is arbitrary, most follow the principles of the rendering equation, and a lot of the time, are just implementations of a BxDF[19].

2.4.1 The Rendering Equation

The rendering equation is[9]:

$$L_o(x, w_o) = L_e(x, w_o) + \int_\Omega f_r(x, w_i, w_o, w_n) L_i(x, w_i) cos\theta_i \, dw_i \tag{1.1}$$

Where:

- L_o represents outgoing light.
- L_e represents emitted light.
- L_i represents incoming light.
- f_r represents the bidirectional distribution function (BxDF).
- x represents the point being shaded.
- w_i represents the direction of incoming light.
- w_o represents the direction of outgoing light.
- w_n represents the direction of the normal at the point x.
- θ_i represents the angle between the normal vector and light source direction.
- Ω represents the hemisphere over which incoming light can potentially reach x.

In simpler terms, the equation is: outgoing light = emitted light + reflected light, and it is used to calculate the colour and intensity of light that is outgoing from a point to the camera. The emitted light term is an arbitrary term that returns what light the primitive emits naturally and can be used to simulate light effects such as fluorescence and phosphorescence. The Integral is used to calculate what is reflected from all incoming light sources to the shaded point. It integrates over all incoming light angles in the points hemisphere, and calculates the product of the BxDF, incoming light at said angle, and a weakening factor due to lambert's law[6][2].

In practice, computers can't calculate infinite integrals, so they settle for one of two approximations. One: sample the point's hemisphere using stochastic (random) sampling; or two: simply sum over a subset of the light sources and primitives visible from the shaded point.

2.4.2 Bidirectional Distribution Functions

There are three main types of bidirectional distribution functions: bidirectional reflectance distribution functions (BRDFs), bidirectional transmittance distribution functions (BTDFs), and bidirectional scattering distribution functions (BSDFs).

A BRDF describes the reflectance properties of the surface for a given pair of incoming and outgoing light directions. In other words, they determine how much light is reflected in a given direction, when light is incident from another direction. BTDFs are similar

[6]Lambert's law states that the intensity of illumination on a diffuse surface is proportional to the cosine of the angle between the surface normal vector and the light source direction

to BRDFs but describe the transmittance properties of the surface, i.e: what light is transmitted through the surface of the material. And a BSDF is a superset, containing both a BRDF and BTDF[20].

Technically, the factor in the rendering equation is a BSDF, but any type of BxDF can be used. BxDFs take as input the incoming and outgoing light direction, as well as the normal of the surface, and other variables relating to the specific implementation of BxDF such as the surface's base colour, metallic value, sheen, and so forth. These light directions and vectors can be seen in figure 2.1.

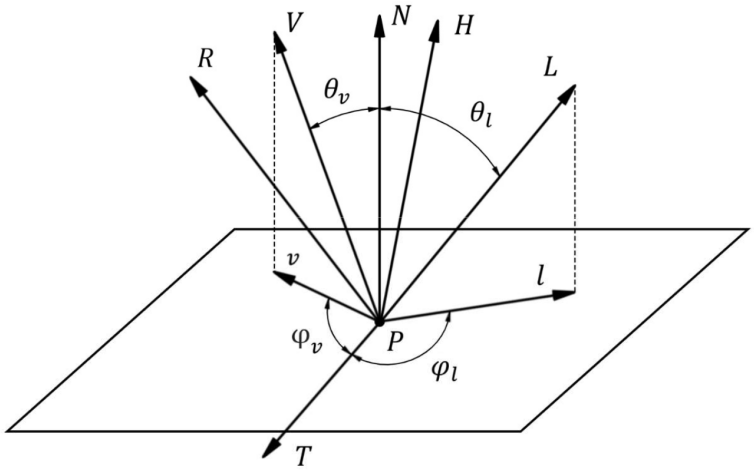


Figure 2.1: Vectors typically used in BxDF evaluation[2]

2.4.3 Physically Based Rendering

In order to make the rendering equation, and BxDFs physically accurate, they should follow the following principles:

- Energy conservation - the incoming energy must equal the outgoing energy.
- Helmholtz reciprocity - the output should be the same if the incoming and outgoing directions are swapped.

There is a whole area of research based around how to make physically based renderers that use BxDFs to simulate micro-facet theory, along with photon mapping or bidirectional path tracing to render photorealistic images. But this is out of the scope of this project, and we only need to understand the rendering equation and BxDFs.

In practice, the only thing that matters is the quality of rendered images and that they look plausible, so even in renderers aiming for photorealism, these principles may or may not be followed.

2.5 Related Work

2.5.1 Wolfenstein 1992

Wolfenstein is a 3D game released in 1992. It uses a combination of ray casting and rasterization to render 3D scenes (now considered 2.5D) with the compute power of 1990s

CPUs.

Each level is stored as a 2D grid, where cells are either solid blocks, or completely empty. Since they knew that each level would be completely encased in blocks, each block would have the same dimensions, and that they would only give players the ability to move the camera horizontally, but never vertically, they could use some clever optimisations to make real-time frame rates feasible.

Ray casting was used to achieve perspective projection, and since each column would necessarily include only one wall tile, along with a ceiling tile and floor tile, only one ray was needed for each column. After the ray hit a wall, offsets were added to the coordinate of the hit point to find the coordinates of the corresponding ceiling and floor. Then, the wall, floor, and ceiling were rasterized, and their textures sampled, for the whole column, at the correct scale for perspective to work. Per-column distance-based darkening, were then used as a crude form of shading.

In Wolfenstein you can see the blueprint for most modern day renderers. There are individual scenes (levels), a hybrid rasterization and ray casting algorithm, texture mapping, and shading. From this you can see where the concepts of scene graphs, programmable shading, and rasterization and ray tracing pipelines came from.

One major shortcoming of this is that it wasn't a general solution, only their hyper-specific level designs could be rendered, and the camera was limited to movement on only one axis. Moreover, scenes were static, and there were no functionality to update them at run-time.

These shortcomings were only there due to the poor processing power of CPUs in the 1990s, and nowadays, CPUs are much more powerful. This project will address these shortcomings, and work to make a general renderer that can render any scenes, so long as there is a camera, and meshes, with arbitrary shading, and these scenes will be able to be updated at run-time.

2.5.2 OpenRT

OpenRT is an open source realtime ray tracing library, primarily designed for academic purposes and physically based rendering. It boasts classical ray tracing; physically correct reflections, refractions, and shadows; and instant global illumination. Its implementation includes: ray-triangle intersection, kd-tree construction and traversal, GPU parallelism, features for handling dynamic scenes, and an OpenRT shader programming interface[19].

Its biggest advantage is its speed, and efficient use of SIMD instructions that run on the GPU. This is also its biggest weakness, however, because it necessarily requires the compute power of GPUs. This project, will try to achieve a form of ray tracing, only using the CPU.

Chapter 3

Design

This Chapter will explain the design of the system from a mid-level view. It will cover the system in terms of what software and tools will be used for the implementation, as well as code structure, class diagrams, and execution flow diagrams. This chapter is based of the finished implementation, rather than an early design that doesn't match the final product, but diagrams are still drawn at a specification level rather than implementation level. This means that functionality that is hyper specific to the implementation are omitted, while still providing more detail than a high-level view.

3.1 System

For the chosen programming language, speed is a necessity since it is essentially trying to run the same code over and over as fast as possible in order to achieve high frame rates. Therefore, high-level languages like python or java won't be any good. What is also important is operator overloading, because the system will be dealing with vector and matrix calculations. Therefore languages that are too low-level like C would make it harder to implement. What is needed is a language that has both speed, and high-level features like operator overloading, and C++ is perfect for this job. C++ also allows both object-oriented programming, and imperative programming, so I can use either paradigm when needed, as well as operator overloading which will be useful for implementing vectors and matrices. For the build system I chose to use CMake.

There is also the choice of what library to use to render pixels to the screen. For this, there are 3 options: a high-level game engine such as unreal engine; accessing the screen directly via the operating system's rendering module; or using a framework/library such as Raylib. Accessing the screen directly via the operating system means it will be too platform specific, and I'll have to either write an abstraction layer for target operating systems, or accept that it will only run on one target operating system. Using high-level engines, like unreal engine, would defeat the purpose of the project because they implement renderers themselves. Consequently, I chose to use Raylib as the rendering library, and I'm only using it to render Bitmaps to the screen - my code will render to a bitmap, that is then rendered to the screen via an OpenGL context that Raylib opens.

3.2 Code Design

Below are explanations of the core modules in the codebase. The class diagrams show the architecture of the codebase, showing the relationship between each class, and how each module interacts with one another.

3.2.1 Engine Module

The engine module, as shown in figure 3.1, is the main module of the codebase. It coordinates between the bitmap used as a frame buffer, the physical window being rendered to, and the scene that is being rendered.

The Engine class has three functions: drawCall(), getScene(), and closeWindow(). The drawCall() function is responsible for doing a full render pass through the pipeline. It clears the bitmap, does geometry processing on the scene, renders said scene to the bitmap, and finally updates the window with the bitmap. The getScene() function is used so that an application using the engine can update the scene in real time, and the closeWindow() function is used so that an application can close down the window before terminating.

The Bitmap3D class consists of a frame buffer and a Z buffer. The setPixel() function bypasses the Z buffer and is used when ray tracing is active since each pixel will only ever be rendered once, so there is no need for visibility determination. The standard drawPixel() function is used for rasterization and does go through the Z buffer.

The Window class is the most complex class of this module. It has its own thread, constantly executing the run() function, that updates the Raylib window with what is stored in its frame buffer, whenever there is an update to the frame buffer. The update() function allows the Engine class to update its frame buffer, and for the window class to tell its thread that a change has been made. The isAlive() function is what allows it to tell its child thread to terminate.

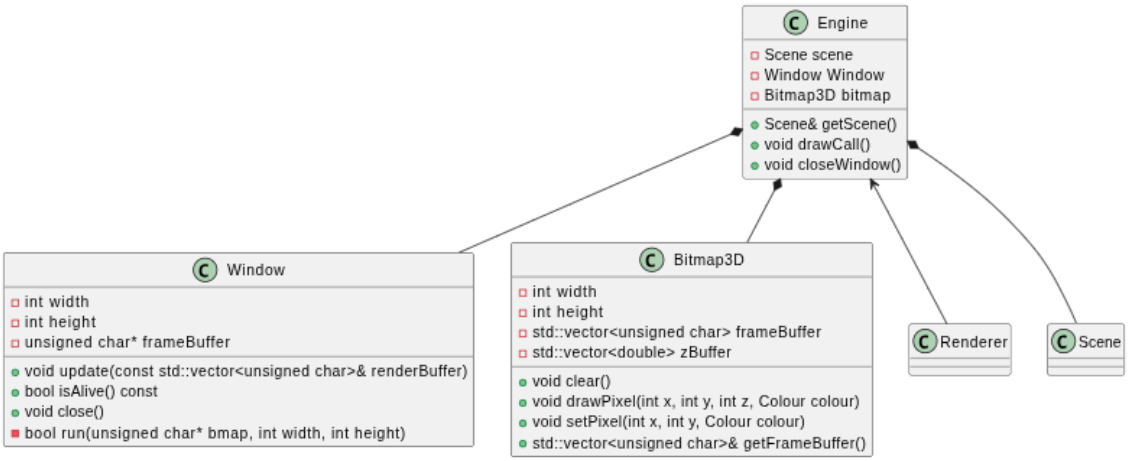


Figure 3.1: Class diagram showing the engine module

3.2.2 Renderer Module

This module, as shown in figure 3.2, contains the Renderer namespace[1], as well as a bounding volume hierarchy class, and related structures.

The Renderer namespace has just two functions for both rendering pipelines implemented: the rasterization pipeline, and the ray tracing pipeline. The functions take in the bitmap to be rendered to, and the scene to be rendered.

The structures that are seen in the module are used in the rasterization and ray tracing algorithms.

A bounding volume hierarchy is a type of acceleration data structure. It is used to speed up the traversal of a scene to find the first triangle that any given ray intersects, if any, and is stored as a binary tree. Further details on how it works can be seen in the implementation chapter.

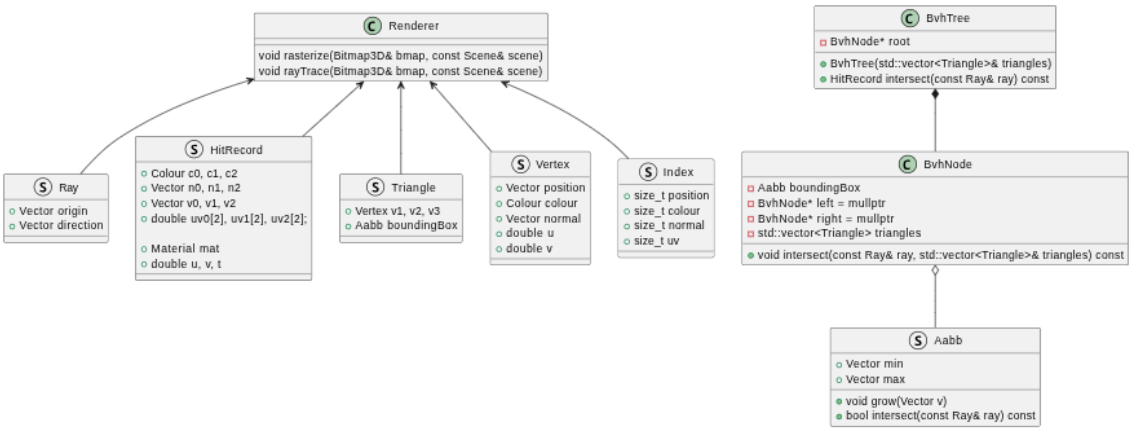


Figure 3.2: Class diagram showing the renderer module

3.2.3 Scene Module

The Scene class, as shown in figure 3.3, contains a set of meshes, lights, and a camera. It has functionality used by the Engine class and Renderer namespace to complete draw calls, and functionality to add more meshes. These meshes are stored as pointers so that the application can modify them in real-time, but once they are added to the Scene class, ownership changes to the scene.

The Mesh class is used to store all the data related to an object; it stores its index and vertex buffers as well as its object-space to world-space transforms. It has functionality to apply transformations to its buffers, reset its buffers to their original state, create an accelerated data structure for efficient ray-mesh traversal, and to perform view-frustum clipping for rasterization.

Each mesh contains a material/shader. This shader is used to determine the colour of a ray once is has intersected a triangle. More materials can be implemented as long as they conform to the specification of the abstract Material class.

The Camera class is used to store all the data related to a virtual camera's position, orientation, focus, and perspective. It has functionality to return matrices used to transform objects from world space to camera space, then clip space, then to viewport space. The algorithms used to create these matrices can be seen in the next chapter.

[1]This is seen as a class in the diagram because PlantUML doesn't allow for functions within namespaces

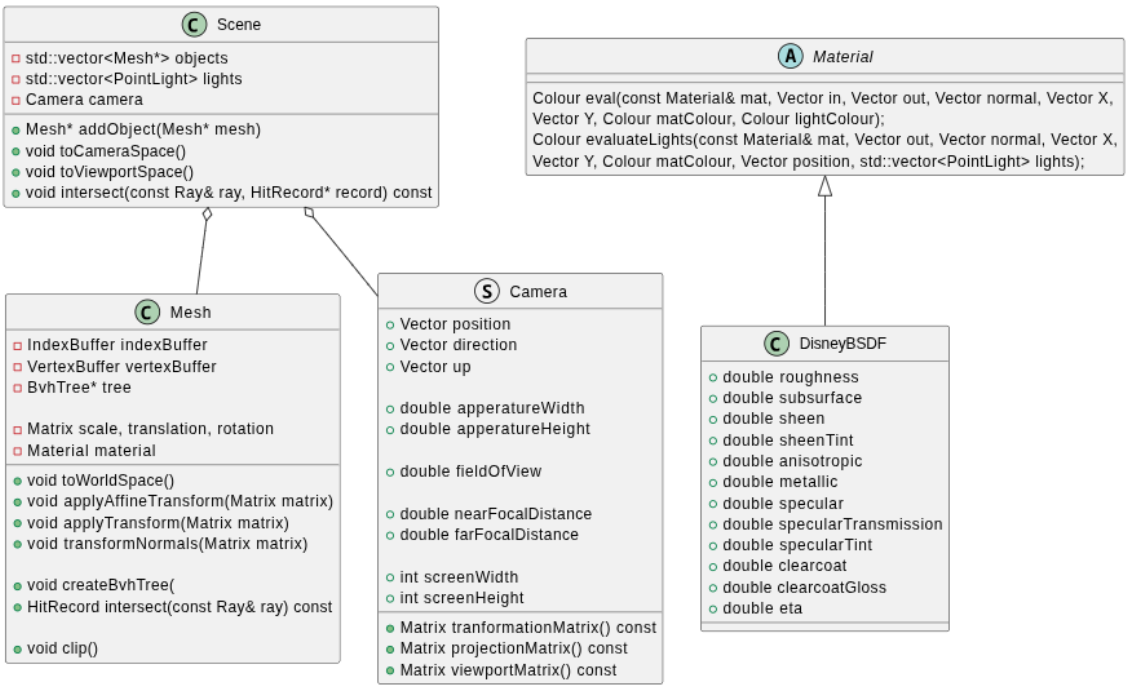


Figure 3.3: Class diagram showing the scene module

3.2.4 Util Module

The util module, as shown in figure 3.4, is the simplest module, which contains the Vector, Matrix, and Colour classes which are used by the rest of the code base.

They have operator overloads for addition, subtraction, multiplication, and division, and static methods for frequently used matrices/vectors and different types of vector multiplication.

Even though vectors and matrices have a theoretically dynamic length (you can have matrices and vectors of any size), that would mean allocating their data on the heap - an area of memory manually managed by the programmer. This is inefficient since you need to go through at least one layer of indirection to access their data. Instead, I gave them both a maximum size, and that is the size that is allocated regardless of the size requested. This means that they can be stored on the stack and access times are, therefore, reduced. This doesn't hamper the usability of the classes, because for graphics purposes, I only need vectors of length 3 or 4, and matrices of size 3x3 or 4x4.

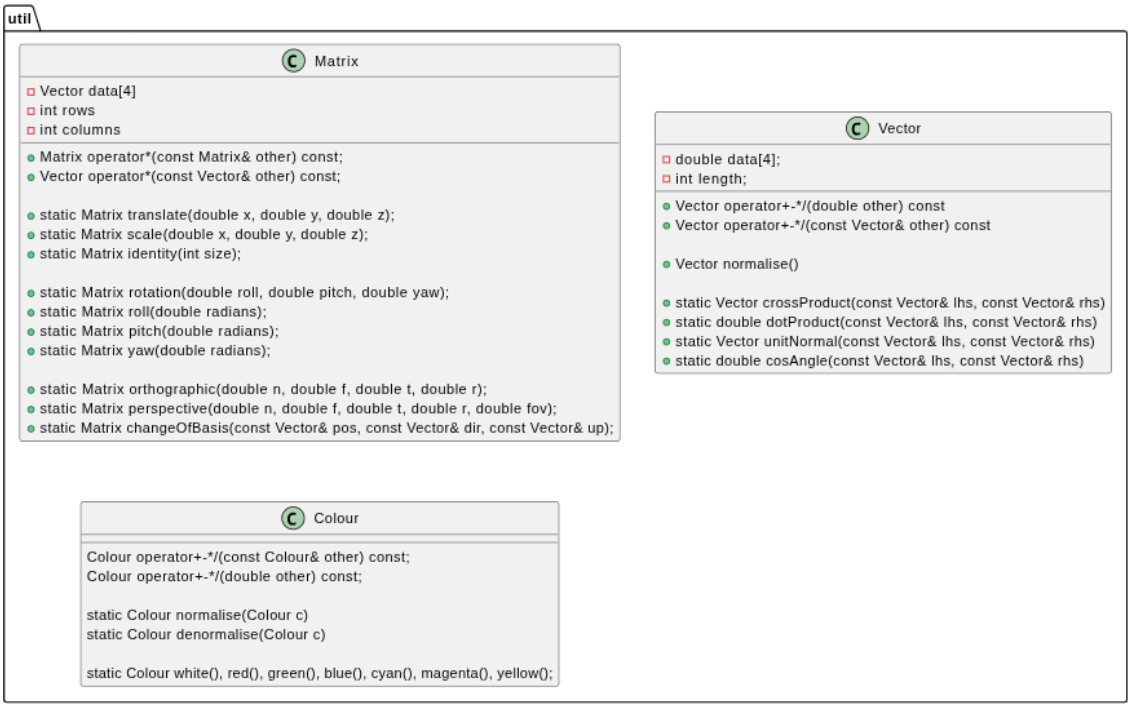


Figure 3.4: Class diagram showing the util module

3.3 Ownership of Data

Each scene stores a set of meshes and cameras. It only stores pointers to its meshes, so that they can be dynamically modified by the application code, and so are stored on the heap. Meshes are instantiated in application code, and registered to a scene. After they are registered to a scene, ownership of said mesh is transferred to the scene object.

Each mesh stores an index buffer and vertex buffer; the vertex buffer stores all the data related to a vertex, including its position, colour, normal, and so forth, and the index buffer will store triangles as indices to the vertex buffer. These buffers are where the geometry processing operations take place, and the two options for this are, one: do all geometry processing operations on a single set of buffers, then revert them at the end of each draw call, or two: copy the buffers every time a draw call is made, then delete them at the end of the draw call.

Since the engine runs on the CPU, processing time is a bigger concern than memory consumption, so the second approach is used. Also, rather than doubling and halving the memory each mesh takes up every draw call, the engine simply stores two copies of each buffer; one copy marked as constant original version of the buffers, and one that is where geometry processing takes place and will be reset each draw call.

Each mesh also stores an acceleration data structure, but since the geometry will change each frame, as the application code updates it, it has to be deleted and regenerated at each draw call. Since acceleration data structures are tree structures, they need to be stored on the heap, and each node stores copies of the vertices within the vertex buffer to make up triangles.

3.4 Code Execution Graph

Figure 3.5 shows the rendering pipeline for a single draw call, showing the order that the code executes, what functions will be called, and what operations will be done. It starts by clearing the bitmap, then transforming all meshes to camera space and doing the ray tracing pass. It then transforms all meshes from camera space to the viewport space and does the rasterization pass. Finally, it resets and clears all buffers and data.

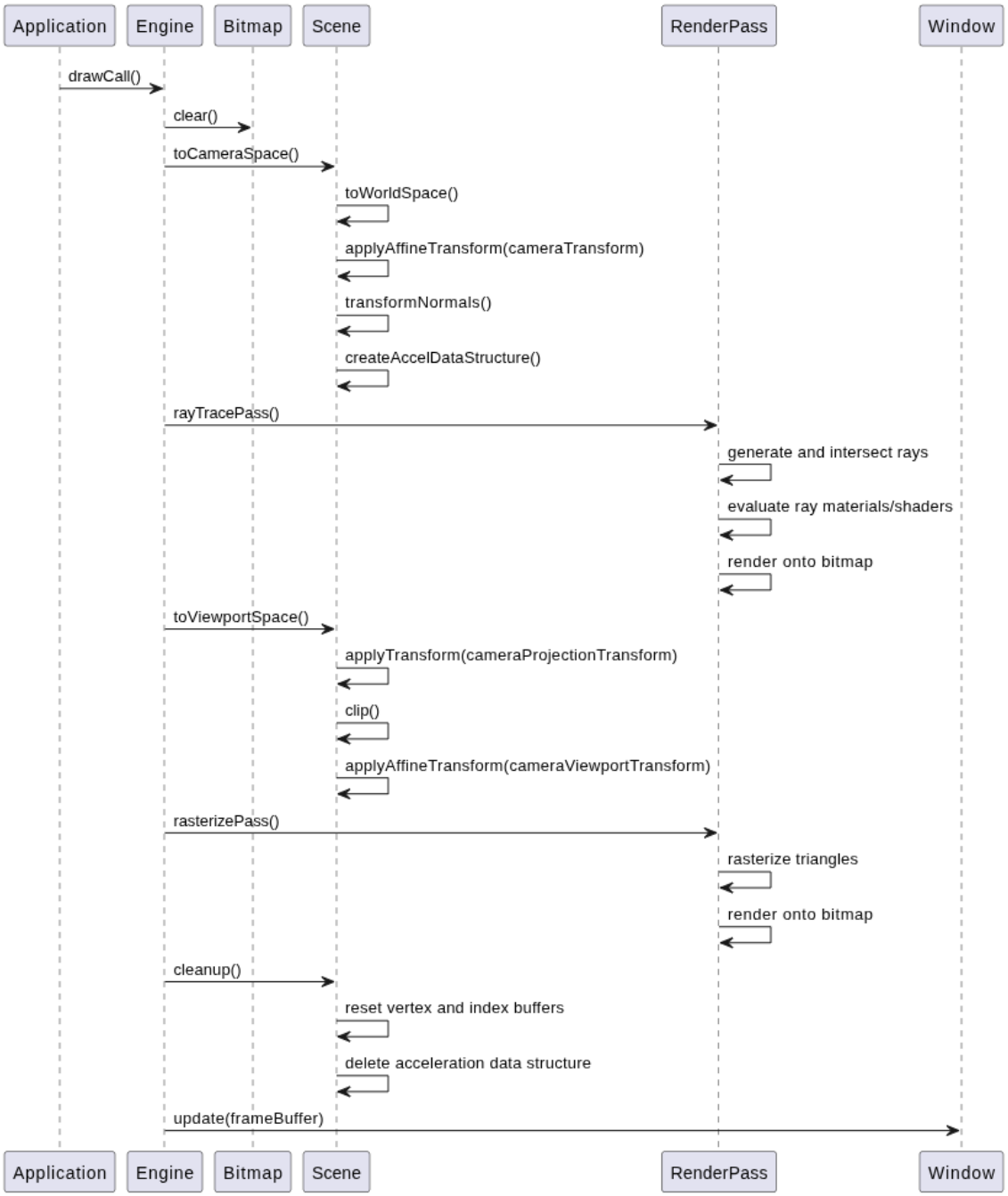


Figure 3.5: Code execution graph showing the execution of a single draw call

Chapter 4

Implementation

This chapter includes the definition and derivation of various matrices implemented in the renderer, explanations of the rasterization pipeline, ray tracing pipeline, and materials interface, as well as pseudo code for their algorithms. Pseudo code for different implementations of certain core modules will also be included, as well as analysis of their strengths and weaknesses, and a final choice will be made for the final versions.

4.1 Matrix Maths & Geometry Processing

Here, the derivations for all transformation matrices will be shown; they are all used to transform meshes into different spaces for rendering. The forms for the simpler matrices will be the same as anywhere, but the more complicated ones are specific to this renderer, and use clever optimisations/tricks to make them more computationally efficient. There are three types of transformations: rigid-body, affine, and homographies. Rigid-body transformations are a subset of affine transformations, and affine transformations are a subset of homographies. Rigid body transformations are the most restrictive, as they preserve lengths, angles and handedness, and consist of only concatenations of translations and rotations. Affine transformations are a step above this, as only the parallelism of lines is preserved, but not necessarily lengths and angles; they consist of a linear transform - any concatenation of rotation, scaling, reflection, and shearing - and then a translation. Finally, there are homographies which deal with projections, and don't even preserve the parallelism of lines[17].

There is more to homographies in that they are non-euclidean transformations, and convert coordinates from a euclidean coordinate space to a homogenous coordinate space. This means that they need to be converted back to euclidean space before rendering, and this is achieved by the w-component/homogenous coordinate. This also means that instead of dealing with 3-element long vectors and 3x3 matrices, we're dealing with 4-element long vectors and 4x4 matrices. After a perspective transform or homography is used, homogenization (division by the w-component) is done to convert back to euclidean space. Note that rigid body and affine transforms don't affect the w-component so homogenization isn't necessary for those.

4.1.1 Transformation Matrix Forms

The Rigid Body Transformation Matrix:

$$R = \begin{bmatrix} r_{00} & r_{01} & r_{02} & t_x \\ r_{10} & r_{11} & r_{12} & t_y \\ r_{20} & r_{21} & r_{22} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} ;\ where\ det(R) = 1\ and\ R^T R = I_4 \tag{4.1}$$

The Affine Transformation Matrix:

$$A = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.2}$$

The Homography Transformation Matrix:

$$H = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{bmatrix} \tag{4.3}$$

4.1.2 Affine Transformations

Note that these transforms are about the origin, so in order to do any one to a mesh, the mesh needs to either already be in object space, or be transformed to the origin, and then back to where it was after the transform is complete. Also, in order to get the expected outcome, they have to be done in a specific order: scaling, then rotating, then translation, so the resultant affine transform is: $A = TRS$.

The Translation Matrix

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.4}$$

The Scaling Matrix

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.5}$$

The Rotation Matrices

$$R_z = \begin{bmatrix} cos(\theta) & -sin(\theta) & 0 & 0 \\ sin(\theta) & cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}; R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & cos(\theta) & -sin(\theta) & 0 \\ 0 & sin(\theta) & cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}; R_y = \begin{bmatrix} cos(\theta) & 0 & sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -sin(\theta) & 0 & cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.6}$$

R_z, R_x, R_y are also known as roll, pitch and yaw respectively.

4.1.3 Projection Transformations

Projection transformations all fall under homographies. The goal of any projection matrix is to transform the scene into the canonical view volume - a cube, centred around the origin, where clipping and rasterization takes place. The cube can have any dimensions, but for this implementation it is (-1, 1) in all three dimensions. For these matrices we will be assuming x is to the right, y is up, and z is forward, though you can easily convert them for any coordinate system by substituting y for -y, if -y is up for example.

Orthographic Projection

The orthographic projection matrix is simply the concatenation of a translation matrix and scaling matrix. The region transformed is defined by two coordinates, one representing the bottom, near, and left walls of the region being transformed, and one representing the top, far, and right walls.

The translation matrix calculates the centre of the cube, and translates this to the origin, and the scaling matrix calculates the correct scale for a width, height, and depth of 2 (-1 to 1).

$$O_{(n,b,l,f,t,r)} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2}{t-b} & 0 & 0 \\ 0 & 0 & \frac{2}{f-n} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -\frac{r+l}{2} \\ 0 & 1 & 0 & -\frac{t+b}{2} \\ 0 & 0 & 1 & -\frac{f+n}{2} \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.7}$$

You can assert left=-right, top=-bottom, to simplify it further, so that you only need near, far, top, and right. The reason you can't do this for near and far, is because once the scene is transformed to camera space (the camera is at the origin looking down the z axis), the origin is only the focal point, and the scene will be some distance along the z-axis where $near \neq -far$. So the final orthographic projection matrix is[7]:

$$O_{(n,f,t,r)} = \begin{bmatrix} \frac{2}{r--r} & 0 & 0 & -\frac{r+-r}{r--r} \\ 0 & \frac{2}{t--t} & 0 & -\frac{t+-t}{t--t} \\ 0 & 0 & \frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{r} & 0 & 0 & 0 \\ 0 & \frac{1}{t} & 0 & 0 \\ 0 & 0 & \frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.8}$$

Perspective Projection

The perspective projection matrix is simply the concatenation of a perspective view matrix, followed by the orthographic projection matrix. The perspective view matrix is to apply perspective down the z-axis, to convert the view frustum to a cube, and then the orthographic projection matrix is used to project this to the canonical view volume. As in the orthographic projection matrix, you can assert left=-right, and top=-bottom[7].

$$P_{(n,f,t,r)} = \begin{bmatrix} \frac{1}{r} & 0 & 0 & 0 \\ 0 & \frac{1}{t} & 0 & 0 \\ 0 & 0 & \frac{2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & f+n & -fn \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} \frac{n}{r} & 0 & 0 & 0 \\ 0 & \frac{n}{t} & 0 & 0 \\ 0 & 0 & \frac{f+n}{f-n} & -2\frac{f*n}{f-n} \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{4.9}$$

A simpler and more versatile solution is to parametrise it with the aspect ratio (a), and vertical field of view (θ), so the fov can be changed at will.

$$P_{(n,f,t,r)} = \begin{bmatrix} \frac{1}{a \cdot tan(\theta/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{tan(\theta/2)} & 0 & 0 \\ 0 & 0 & \frac{f+n}{f-n} & -2\frac{f*n}{f-n} \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{4.10}$$

4.1.4 Other Transformations

The Camera Transform

The camera transform is used to orient the camera at the origin looking down the positive z axis, with up being in the positive y direction, while moving the whole scene along with it. It is a concatenation of a translation matrix, used to transform the scene to the origin, and a change of basis matrix, used to orient the scene.

The change of basis matrix is defined by a position coordinate, direction vector, and up vector. The negative normalised cross product of the direction and up vectors is used to find the right basis, and the cross product of the direction and right basis is used to find the up basis. The right basis, up basis, and direction are 3-element vectors and are used to form the change of basis matrix.

The final camera transform is defined by the right basis, up basis, direction, and position vectors/coordinates.

$$C = \begin{bmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ d_x & d_y & d_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -p_x \\ 0 & 1 & 0 & -p_y \\ 0 & 0 & 1 & -p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} r_x & r_y & r_z & -p \cdot r \\ u_x & u_y & u_z & -p \cdot u \\ d_x & d_y & d_z & -p \cdot v \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.11}$$

The Viewport Transform

The viewport transform, like the orthographic projection transform, is just the concatenation of a scaling and translation matrix, but this time in the other direction. It scales it to the width and height of the viewport, where -y is up, then translates it so that the origin is at the top left point of the viewport. 10000 is used for the scale of the z axis so that there is no loss in precision for floating points - it's also translated by this value so that its range is (0, 10000), as the bitmap expects it, rather than (-5000, 5000).

$$V_{(w,h)} = \begin{bmatrix} 1 & 0 & 0 & \frac{w}{2} \\ 0 & 1 & 0 & \frac{h}{2} \\ 0 & 0 & 1 & 10000 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{w}{2} & 0 & 0 & 0 \\ 0 & -\frac{h}{2} & 0 & 0 \\ 0 & 0 & 10000 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{w}{2} & 0 & 0 & \frac{w}{2} \\ 0 & -\frac{h}{2} & 0 & \frac{h}{2} \\ 0 & 0 & 10000 & 10000 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.12}$$

The Normal Transform

Normals can be recalculated after transformations have taken place, but it is often easier to translate them with the vertices.

The most correct general method is to use the inverse-transpose of the transformation matrix instead, but this doesn't work when the determinant of the original matrix is zero, as it requires dividing the adjoint by the determinant. Hence the adjoint-transpose of the transformation matrix is used instead, with renormalisation afterwards, since the adjoint-transpose doesn't guarantee the result to be of unit length. This is a costly calculation, however, and we can speed it up by applying a few constraints to the renderer.

Firstly, since the normal is only used for shading, and shading is done when the scene is in camera space, we can assume that only affine transformations will have taken place. This means that the w-component of each vertex will be left unchanged at 1, and instead of calculating the adjoint-transpose of the 4x4 transformation matrix, we can calculate it from the upper left 3x3 matrix within it.

Furthermore, for this renderer, only rotation, scaling, and translation operations are allowed. Translation and scaling don't affect the normal, so what is left is only two rotations, one to get from object space to world space, and one to get from world space to camera space. We can use the fact that a rotation matrix's transpose is its inverse, so the inverse-transpose of a rotation matrix is simply the rotation matrix itself - and this avoids the downside of using the adjoint-transpose instead.

Therefore, whenever a rotation is applied to a mesh, it is also applied to its normals.

4.1.5 The Geometry Processing Pipeline

The geometry processing pipeline, as shown in figure 4.1, is the pipeline that converts meshes from object space to camera space, and then from camera space to viewport space, going through all the coordinate space in between, parametrised by the cameras position and orientation, and the meshes' affine transform.

From object space to camera space:

```
void Scene::toCameraSpace() {
    for (Mesh* mesh : objects) {
        mesh->toWorldSpace();
        mesh->applyAffineTransform(camera.tranformationMatrix());
        mesh->transformNormals(camera.rotationMatrix());
        mesh->createAccelDataStrucutre();
    }
    for (PointLight light : lights) {
        light.applyAffineTransform(camera.tranformationMatrix());
    }
}
```

This is where ray tracing takes place, and the affine transformation matrix used in toWorldSpace() is stored as well as its individual translation, scaling, and rotation matrices, so that it doesn't need to be recalculated every time toWorldSpace() is called.

From camera space to viewport space:

```
void Scene::toViewportSpace() {
    for (Mesh* mesh : objects) {
```

```
            mesh->applyTransform(camera.projectionMatrix());
            mesh->clip();
            mesh->applyAffineTransform(camera.viewportMatrix());
        }
    }
```

This is where rasterization takes place, and the clip() function performs view frustrum clipping, where all triangles that are outside of the canonical view volume are 'clipped' such that they aren't rendered, since they are outside of what will be seen.

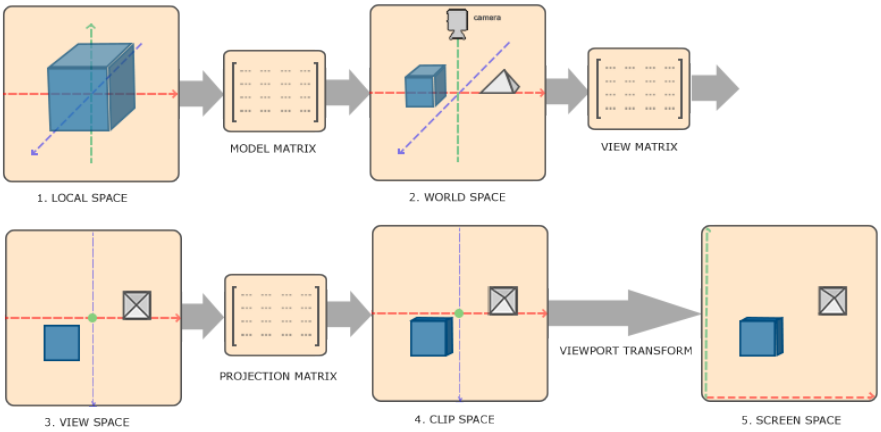


Figure 4.1: Graph showing the geometry processing pipeline for a mesh[6]

4.2 The Rendering Pipeline

4.2.1 Ray Tracing Algorithm

The ray tracing algorithm is run once the scene is converted to camera space. It assumes that the camera is positioned at the origin, and looking down the z axis, with up being along the y axis. It generates normalised vector rays for each pixel, intersects them with the scene, then does colour determination with the materials interface that is explained in 4.3: Shading.

The ray-scene intersection can be done in any way as long as it returns a record of the hit, including the following information: the colour, normal, and position of each vertex of the hit triangle, the material of the triangle, as well as the barycentric coordinates and t value of the hit point.

The main algorithms to do this are the primary ray generation algorithm, and the ray-scene and ray-triangle intersection algorithms. This implementation uses a combination of bounding volume hierarchy intersection, aabb intersection, and Moller-Trumbore ray-triangle intersection, and uses the barycentric coordinates for interpolating the colour, normal, and position of the hit point. The colour, normal, and position is then used in BxDF shading to find the final colour of the pixel.

Rays are defined by an origin, or start point, and direction. Consequently, any point along the ray is defined as[12]:

$$R(t) = O + tD \tag{4.13}$$

Where t represents a distance along its direction.

Below is the main algorithm of the ray tracing pipeline:

```
void Renderer::rayTrace(Bitmap3D& bmap, const Scene& scene) {
    const Camera& camera = scene.getCam();
    const int x = camera.screenWidth/2;
    const int y = camera.screenHeight/2;

    //loop through each pixel of the window
    for (int i = -x; i < x; i++) {
        for (int j = -y; j < y; j++) {
            Ray ray = Ray(
                Vector(0, 0, 0),
                Vector(i, j, camera.nearFocalDistance).normalise()
            );

            HitRecord record;
            record.t = DOUBLE_MAX;
            scene.intersect(ray, record);

            if (record.t != DOUBLE_MAX) {
                Colour c = Mat::eval(record);
                bmap.setPixel(i+x, camera.screenHeight-(j+y), c);
            }
        }
    }
}
```

Bounding Volume Hierarchies

The renderer uses bounding volume hierarchies as its acceleration data structures, and it uses them on a per mesh basis, with separate independent BVHs per mesh. This means that for ray-scene intersection, each mesh in the scene is searched through linearly, while the triangles of each mesh are searched through logarithmically with the BVH traversal algorithm.

The BVH creation algorithm works by precomputing the aabb of each triangle, then recursively, for each node, calculating the containing aabb over all the triangles in the node, finding the optimal split index, and splitting the triangles into two sets, one for each child node. If less than or equal to the minimum number of triangles per node is met, then that node becomes a leaf node, storing copies of the mesh's triangles. Triangle-median is used as the splitting criteria, where the longest axis of the containing aabb is found, and the triangles are ordered by their position along this axis. Then, the median index of these triangles is found, and that is used as the splitting index.

One thing to note is that the whole array of triangles does not need to be fully sorted for triangle-median splitting to work. The only requirement is that all the triangles before the split index have a lower position in the split axis than all the triangles after the split index. This brings the average time complexity of a full sort from $O(n\,log\,n)$ to $O(n)$, and the average time complexity of the creation algorithm from $O(n\,log^2\,n)$ to $O(n\,log\,n)$[10].

BVH traversal works by creating a buffer, where candidate triangles will be stored. This buffer is passed by reference to the intersection function of the root node of the BVH, along with the ray. Any given node will return if the ray does not intersect with its containing aabb, and add its triangles to the buffer if it is a leaf node, and otherwise recursively call the intersect functions of its child nodes. After all candidate triangles are found, Moller-Trumbore intersection is used to find the closest intersection, and the data of this triangle is added to the hit record which is returned[8].

This can be done with a single BVHNode class, but it is often easier to have a separate BVHTree class that handles the creation logic, and creates the hit record after intersection has completed. The BVHTree class stores the root node, and the BVHNode class stores the containing aabb, and either left and right pointers or a vector of triangles.

Below is the code for BVH creation and traversal:

```
// creates a whole BVH
BvhTree::BvhTree(std::vector<Triangle>& tris) {
    computeBoundingBoxes(tris);
    root = new BvhNode(tris, 0, tris.size()-1);
}

// creates a BVH node, and its sub nodes
BvhNode::BvhNode(std::vector<Triangle>& tris, size_t start, size_t end) {
    // step 0 - calculate containing bbox
    for (int i = start; i <= end; i++) {
        boundingBox.grow(tris[i].boundingBox);
    }

    // step 1 - stop recursing if minimal triangles is met
    int range = (end-start) + 1;
    if (range < 2 * MIN_TRIANGLES) {
        for (int i = start; i <= end; i++)
            triangles.push_back(tris[i]);

        return;
    }

    // step 2 - calculate optimal split, then order triangles
    Vector span = boundingBox.max - boundingBox.min;

    short axis = 0;
    if (span.y() > span[axis]) axis = 1;
    if (span.z() > span[axis]) axis = 2;
    int splitIndex = (end-start) / 2;

    nthElement(tris, splitIndex, axis);

    // step 3 - recurse
    left = new BvhNode(tris, start, start+splitIndex);
```

```
    right = new BvhNode(tris, start+splitIndex+1, end);
}

// tree intersection
HitRecord BvhTree::intersect(const Ray& ray) const {
    // find and reutrn all candidate triangles
    std::vector<Triangle> triangles{};
    root->intersect(ray, triangles);
    return closestIntersection(triangles);
}

// node intersection
void BvhNode::intersect(const Ray& ray, std::vector<Triangle>& tris) const {
    if (!boundingBox.intersect(ray))
        return;

    // if leaf node
    if (!triangles.empty()) {
        tris.insert(tris.end(), triangles.begin(), triangles.end());
        return;
    }

    // if non-leaf node
    if (left) left->intersect(ray, tris);
    if (right) right->intersect(ray, tris);
}
```

Ray-aabb Intersection

Ray-aabb intersection is done during the traversal of a bvh, and is used to determine if a ray intersects with any given node. An aabb is defined by a start coordinate and end coordinate, or alternatively: 6 planes, 2 in each axis that form the walls of the cube, or alternatively again: by 3 slabs in the 3 axis directions defined by 2 planes each.

The intersection algorithm works as follows, first the t-intervals that the ray intersects each plane is calculated. They are then ordered in each axis so that you have a t value where the ray enters each slab and exits each slab. If these 3 t-intervals overlap, then that means the ray intersects the aabb at some point[13].

Below is the intersection algorithm implemented:

```
bool Aabb::intersect(const Ray& ray) const {
    //calculate the t intervals
    Vector t0 = (min - ray.origin) / ray.direction;
    Vector t1 = (max - ray.origin) / ray.direction;

    //order the t intervals
    if (t0.x() > t1.x()) std::swap(t0.x(), t1.x());
    if (t0.y() > t1.y()) std::swap(t0.y(), t1.y());
```

```
    if (t0.z() > t1.z()) std::swap(t0.z(), t1.z());

    //if the intervals of x and y don't match, return false
    t0.x() = t0.x() > t0.y() ? t0.x() : t0.y();
    t1.x() = t1.x() < t1.y() ? t1.x() : t1.y();
    if (t1.x() <= t0.x()) return false;

    //if the intervals of x, y, and z don't match, return false
    t0.x() = t0.x() > t0.z() ? t0.x() : t0.z();
    t1.x() = t1.x() < t1.z() ? t1.x() : t1.z();
    if (t1.x() <= t0.x()) return false;

    //it is a hit
    return true;
}
```

Barycentric Coordinates

Barycentric coordinates allow use to express the position of any point on/in a triangle by three scaler factors: u, v, and w. A point, represented in barycentric form is defined as:

$$P = wV_0 + uV_1 + vV_2 \tag{4.14}$$

Where V_0, V_1, V_2 are the vertices of the triangle and: $w + u + v = 1$. Meaning you only really need two scaler factors u and v to express any barycentric point. w, u, and v are calculated from the proportion of the area of the full triangle that the three sub-triangles defined by the point P and vertices V_0, V_1, V_2 make up.

$$P = (1 - u - v)V_0 + uV_1 + vV_2 \tag{4.15}$$

These coordinates are used for interpolating the colour, normal and position of the hit point between the three vertices of a hit triangle, since colour, normal, and position values are stored per-vertex. This is done using the fact that u, v, and w, are values normalised between 0 and 1 that represent how close they are to each vertex. Below is the equation used to interpolate the colour of the hit point of a triangle[14].

$$P_c = V0_c \cdot w + V1_c \cdot u + V2_c \cdot v \tag{4.16}$$

Moller-Trumbore Intersection

The Moller-Trumbore intersection algorithm is used to determine whether a ray intersects with a triangle or not. It is memory efficient as it does not require any additional storage, or any precomputaion of normals. It returns the distance to the intersection point t, and the barycentric coordinates of the intersection point u, v[11].

It works by equating the equation of a ray to the equation of a point on a triangle:

$$R(t) = O + tD = (1 - u - v)V_0 + uV_1 + vV_2 = T(u, v) \tag{4.17}$$

Rearanging to get:

$$\begin{bmatrix} -D & V_1 - V_0 & V_2 - V_0 \end{bmatrix} \begin{bmatrix} t \\ u \\ v \end{bmatrix} = O - V_0 \tag{4.18}$$

And solving for [t, u, v] to get:

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{(D \times E_2) \cdot E_1} \begin{bmatrix} (T \times E_1) \cdot E_2 \\ (D \times E_2) \cdot T \\ (T \times E_1) \cdot D \end{bmatrix} \tag{4.19}$$

Where $E_1 = V_1 - V_0$, $E_2 = V_2 - V_0$, and $T = O - V_0$

The version implemented also does backface culling, where all triangles that are facing away from the ray that hits it are ignored. This is done by comparing the determinant to epsilon - technically comparing it to zero is correct, but epsilon is used for numerical stability. This also means that we can delay dividing by the determinant until an intersection is confirmed, meaning that the upper bound for u and v is the *determinant* $\times$ 1 instead of just 1.

Below is the intersection algorithm implemented:

```
double BvhTree::mollerTrumboreIntersection(
    const Ray& ray,
    const Vector& v0, const Vector& v1, const Vector& v2,
    double& u, double& v
)
{
    constexpr double epsilon = std::numeric_limits<double>::epsilon();

    Vector edge1 = v1 - v0;
    Vector edge2 = v2 - v0;

    Vector pvec = Vector::crossProduct(ray.direction, edge2);
    double det = Vector::dotProduct(edge1, pvec);

    // back face culling check
    if (det < epsilon)
        return -1;

    Vector tvec = ray.origin - v0;
    u = Vector::dotProduct(tvec, pvec);

    // check u is within the triangle
    if (u < 0.0 || u > det)
        return -1;

    Vector qvec = Vector::crossProduct(tvec, edge1);
    v = Vector::dotProduct(ray.direction, qvec);

    // check v is within the triangle
    if (v < 0.0 || u + v > det)
        return -1;

    // can now calculate t, u, v
```

```
    double t = Vector::dotProduct(edge2, qvec);
    double invDet = 1.0 / det;
    t *= invDet;
    u *= invDet;
    v *= invDet;

    return t;
}
```

4.2.2 Raster Algorithm

The rasterization algorithm is implemented using a scanline algorithm to convert each triangle into lines, and a digital differential analyser algorithm to rasterize each line into the frame buffer. These algorithms together work on a per triangle basis, so each triangle is rendered independent of one another. The Z buffer is responsible for visibility resolution.

The scanline algorithm works by calculating the gradient for each edge of the triangle, going from left to right, and by calculating the difference in x from the leftmost vertex (v1) to the middle vertex (v2), and from the middle vertex to the rightmost vertex (v3). It then loops through each x from v1 to v2, and then from v2 to v3, using the gradients previously calculated to find the corresponding y, and z values, yielding a set of vertical lines, defined by their start and end coordinates. Gradients are also calculated for the red, green, blue, alpha components of the colour of each vertex, and this is used for a naive version of shading - the ray tracer does more complicated shading. One thing to note is that for this algorithm to work properly, the vertex ordering of each triangle must be consistent, either clockwise or anticlockwise. Therefore, at the start of this algorithm the vertices are sorted into a clockwise vertex ordering.

These lines are then rasterized using the digital differential analyser algorithm (DDA). The DDA algorithm works by interpolating variables over an interval between start and end points. It interpolates the x, y, z, and r, g, b, a values of the start point and end point over the length of each line. First it determines which axis, x or y, has the greatest change, and then works out the gradient, and error for each variable that is interpolated. Finally, the line equations made up of these gradients are looped over, incrementing the axis that has the greatest change for maximum accuracy, rasterizing each point of the line into the frame buffer.

Pseudo code for the rasterization algorithm; it only contains the interpolation of x, y, and z in order to save space:

```
void rasterize(Bitmap3D bmap, Scene scene) {
    foreach (mesh m in scene
        foreach (triangle t in m)
            drawTriangle(bmap, t.v1, t.v2, t.v3);
}

void drawTriangle(Bitmap3D bmap, v1, v2, v3) {
    // orders the vertices in a clockwise winding
    orderVertices(v1, v2, v3);

    // calculates the gradients and differences of each line
```

```
    Vector v1ToV2 = v2-v1, v1ToV3 = v3-v1, v2ToV3 = v3-v2;
    Vector dBydx12 = v1ToV2 / v1ToV2.x();
    Vector dBydx13 = v1ToV3 / v1ToV3.x();
    Vector dBydx23 = v2ToV3 / v2ToV3.x();

    // loop through each x rastering each line
    for (double x = 0; x < v1ToV2.x(); x++) {
        Vector vertex1 = v1 + Vector(x, dBydx12.y() * x, dBydx12.z() * x);
        Vector vertex2 = v1 + Vector(x, dBydx13.y() * x, dBydx13.z() * x);
        drawLine(bmap, vertex1, vertex2);
    }
    for (double x = 0; x < v1ToV2.x(); x++) {
        double x2 = v1ToV2.x() + x;
        Vector vertex1 = v2 + Vector(x, dBydx23.y() * x, dBydx23.z() * x, 0);
        Vector vertex2 = v1 + Vector(x2, dBydx13.y() * x2, dBydx13.z() * x2, 0);
        drawLine(bmap, vertex1, vertex2);
    }
}

void drawLine(Bitmap3D bmap, v1, v2) {
    // calculates the differences of the line for each axis
    int x1 = start.x(), y1 = start.y(), z1 = start.z();
    int x2 = end.x(), y2 = end.y(), z2 = end.z();
    double dx = x2-x1, dy = y2-y1, dz = z2-z1;

    if (abs(dx) < abs(dy)) {
        if (dx == 0) return;

        // variables for the line equations
        double ySlope = dy / dx;
        double zSlope = dz / dx;
        int yError = (y1 - ySlope * x1);
        int zError = (z1 - zSlope * x1);

        // loops through each x rasterizing each pixel
        int xStep = dx < 0 ? -1 : 1;
        while (x1 != x2) {
            x1 += xStep;
            y1 = (ySlope * x1) + yError;
            z1 = (zSlope * x1) + zError;
            c1 = ...
            bmap.drawPixel(x1, y1, z1, c1);
        }
    }
    else {
        // the same but using yStep instead of xStep.
    }
}
```

4.3 Shading

4.3.1 Materials Interface

The materials interface is used to make a single interface that all shaders use. This abstracts shaders away from the rest of the codebase so that shaders can be programmed independently, as long as they follow the rules of the interface. The interface will only include one function: the BxDF function seen in the rendering equation in the background chapter. Before settling on the final version, I went through three different implementations of the materials interface: class based polymorphism, union based polymorphism, and variant based polymorphism.

Below contains the code that was used for the three different versions.

Class Based Polymorphism

Class based polymorphism would require having a base abstract 'Material' class that stores a virtual function for evaluating a material/shader, and each derived class will implement this function, and store any parameters used in the evaluation.

Then, each mesh will store a pointer of this abstract class, that can point to any material that implements it. This has the downside of going through one layer of indirection - the abstract class - before reaching a derived class, and would also need to be stored on the heap. Below shows what this would look like implemented in C++.

This is the first version I tried, but because of the above limitations I changed it to union based polymorphism.

```
// base material class, containing the evalute method
class Material {
public:
    virtual
    Colour
    evaluate(Vector& in, Vector& out, Vector& normal, Colour& colour) = 0;
};

// derived class representing an implemented material
class Material1 : public Material {
public:
    double param1{};
    double param2{};
    double param3{};

    Colour
    evaluate(Vector& i, Vector& o, Vector& n, Colour& c) override {
        ...
    }
};

// how a material would be stored within the mesh class
class Mesh {
```

```
private:
    ...
    Material* mat = Material1{};
    ...
};
```

Union Based Polymorphism

Union based polymorphism would fix the issues of storing materials on the heap, and is the fastest method available, with as few abstractions as possible, but requires the most effort to manage. Since when adding a new material: the 'MaterialType' enum, would need to be updated; a new struct for the material needs to be implemented, along with the 'evaluate' function with the correct function signature; it needs to added to the 'Material' structs union; and finally it needs to be added to the switch statement within the main evaluate function.

This is the second version I tried, but because of the above limitations, and the advantages of variant based polymorphism, I didn't go for it.

```
// enum storing a state for each material
enum MaterialType {
    MATERIAL_1,
};

// struct representing an implemented material
struct Material1 {
    double param1{};
    double param2{};
    double param3{};

    Colour evaluate(Vector& i, Vector& o, Vector& n, Colour& c) {
        ...
    }
} typedef Material1;

// main material struct that can contain any concrete material
struct Material {
    MaterialType flag;
    union {
        Material1 material1;
    };

    // main evalute method that calls the correct materials evaluate
    // method
    Colour evaluate(Vector& i, Vector& o, Vector& n, Colour& c) {
        switch (flag) {
            case MATERIAL_1: {
                material1.evaluate(i, o, n, c);
                break;
```

```
            }
        }
    }
};
```

Variant Based Polymorphism

Variant based polymorphism abstracts away the union handling of union based polymorphism, but works in the same way under the hood. It makes the code more manageable while keeping it only marginally less efficient. In order to implement a new material, all that needs to be added is the materials struct, a function definition in the 'evaluate' struct, and for it to be registered in the 'Material' variant.

This is the final version I settled on, for its combined efficiency and ease of use.

```
// struct representing an implemented material
struct Material1 {
    double param1{};
    double param2{};
    double param3{};
} typedef Material1;

// variant responsible for polymorphism
typedef std::variant<Material1, ...> Material;

// visitor struct for calling the evaluate method
struct evaluate {
    const Vector& in;
    const Vector& out;
    const Vector& normal;
    const Colour& colour;

    Colour operator()(const Material1& material) const;
    ...
};

// implementation of the evaluate method for Material1
Colour evaluate::operator()(const Material1& material) const {
    ...
}

// main evalute function that uses std::visit to call the correct
// evaluate method
Colour
eval(const Material& mat, Vector& n, Vector& t, Vector& n, Colour& c) {
    return std::visit( Mat::evaluate{i, o, n, c}, mat);
}
```

4.3.2 The Rendering Equation Algorithm

Here is the rendering equation again for your convenience:

$$L_o(x, w_o) = L_e(x, w_o) + \int_\Omega f_r(x, w_i, w_o, w_n) L_i(x, w_i) cos\theta_i \, dw_i \tag{4.20}$$

The shading algorithm for a point hit by a ray works by calculating the integral of the rendering equation for each light source in the scene, and averaging them - only the reflected light is calculated, the emitted light term is ignored. Then, exposure scaling is applied to the resultant colour, to get the average brightness of the scene to a reasonable level; followed by Reinhard tone mapping to normalise it between [0, 1]; followed by multiplying it by 255, to get it to the correct rgb range of [0, 255]. Finally, the alpha value is set to 255 because semi-transparent materials, if implemented, wouldn't be solved by the rendering equation.

The inputs to the rendering equation solver are the material, in, out, normal and position vectors representing w_i, w_o, w_n, and x respectively, the orthogonal basis vectors X and Y, as well as the base colour of the material, and the light sources.

The material, in, out, normal, position, and material colour are all known from ray-scene intersection, while the light sources are known beforehand, but the X, and Y orthogonal basis are not immediately known. They are defined as a pair of vectors representing the axis of the plane that the hit triangle lies on, and are perpendicular to each other and the normal vector. We can use the fact the the cross product of two vectors results in a vector that is perpendicular to both the input vectors to find out the basis vectors. It is done by choosing a random vector that is not the normal, the finding the cross product between this and the normal - this is the X vector. Then we find the cross product between X and the normal to find Y.

Below is the implementation for solving the rendering equation for a point hit by a ray for multiple light sources:

```
Colour Mat::evaluateLights(
    const Material& mat, Vector out, Vector normal,
    Vector X, Vector Y, Colour normalisedMatColour,
    Vector position, std::vector<PointLight> lights
)
{
    Colour colourSum = Colour{0, 0, 0, 0};

    for (const PointLight& L : lights) {
        Colour lightColour = Colour::normalise(L.colour);
        Vector in = (L.position - position).normalise();

        colourSum = colourSum + Mat::eval(
            mat, in, out, normal, X, Y,
            normalisedMatColour, lightColour
        );
    }
```

```
    colourSum = colourSum / lights.size();
    Colour colour = colourSum * std::pow(2, EXPOSURE);
    colour = colour / (colour + 1);
    colour = Colour::denormalise(colour);
    colour.a() = 255;

    return colour;
}

Colour Mat::eval(
    const Material& mat, Vector in, Vector out,
    Vector normal, Vector X, Vector Y,
    Colour normalisedMatColour, Colour normalisedlightColour
)
{
    double lambert = std::max(0.0, Vector::dotProduct(in, normal));

    Colour bxdf = std::visit(
        Mat::evaluateBxDF{in, out, normal, X, Y, normalisedMatColour},
        mat
    );

    return normalisedlightColour * bxdf * lambert;
}
```

4.4 Miscellaneous

4.4.1 Reading and Writing Data

There is functionality for reading in mesh data in the form of wavefront (.obj) files. This includes the normals, positions, and UV coordinates of all vertices for a mesh - wavefront files don't store colour data. This can be done from application code when registering meshes to the renderers scene.

Also, there is functionality for writing each frame to storage as a Portable PixMap (.ppm) file, allowing the user to save rendered frames, or render a full video.

4.4.2 Command Line Interface

There are several ways to run the engine from the command line. The first is without any additional arguments, and a default scene is rendered. The second is with the name of one of the hardcoded scenes and a flag signalling which rendering pipeline to use. The third is with the filename of a .obj file, the width and height of the screen, and the flag signalling which pipeline to use. The third method will run the speed test code, rendering the scene for 10 frames while rotating the mesh, and display the overall frame rate and time period.

1. ./bin/ChessEngine

2. ./bin/Win3D [meshPath] [width] [height] [renderType]
3. ./bin/Win3D [scene] [renderType]

Chapter 5

Testing and Evaluation

This chapter will explain the tests run on the renderer, show their results, evaluate what they mean, as well as give final renderers of the engine.

For the rasterization and ray tracing pipelines, they were both run with, one: a varying number of triangles in the scene, while keeping a constant resolution, and two: with a varying resolution/pixel count, while keeping the number of triangles constant. Each scene ran for 10 frames, and the total time of the scene was tracked - this was used to calculate the frames per second (fps) and time period. This data was plotted onto line graphs and the results can be seen below.

The engine was tested on a laptop with an Intel Core i7-1255u @ 4.70 GHz with 12MB cache.

5.1 Rasterization Pipeline

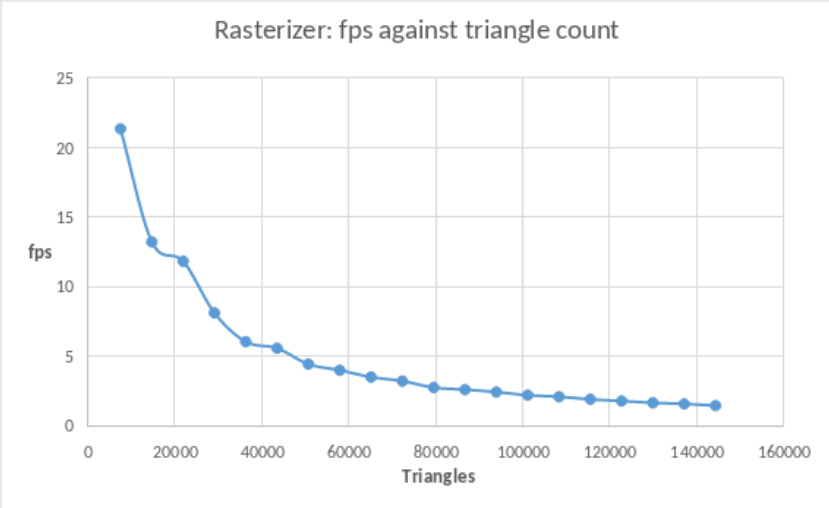


Figure 5.1: Shows the frames per second achieved by the rasterizer when rendering different amounts of triangles

Figure 5.1 shows that the time period (1/fps) for one frame to render has a logarithmic relationship to the number of triangles in a scene.

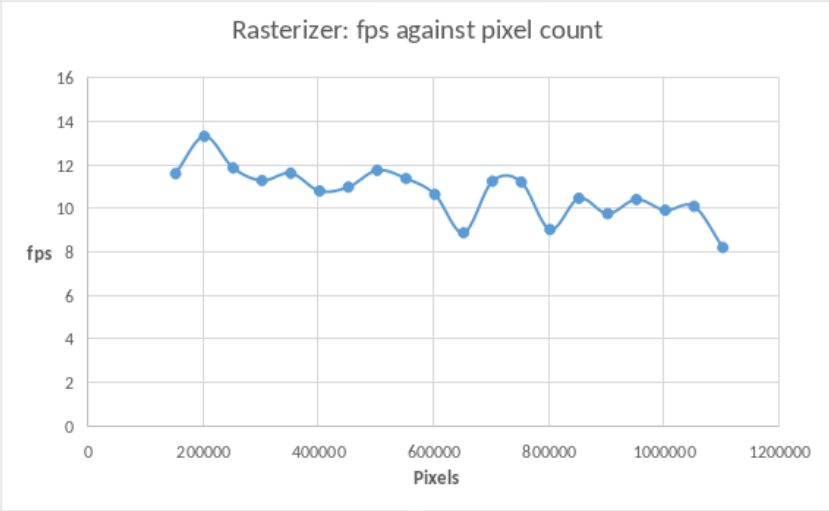


Figure 5.2: Shows the frames per second achieved by the rasterizer when rendering at different resolutions

Figure 5.2 shows that the fps has a slight downward trend as the resolution increases. The time for the raster algorithm to complete does not increase with resolution, however. The reason for the decrease in speed is that operations on a frame buffer get inherently slower as its size increases. One thing to note is that this graph has a higher random variation between data points than the triangle count graph. This is probably due to random variation/fluctuations.

5.2 Ray Tracing Pipeline

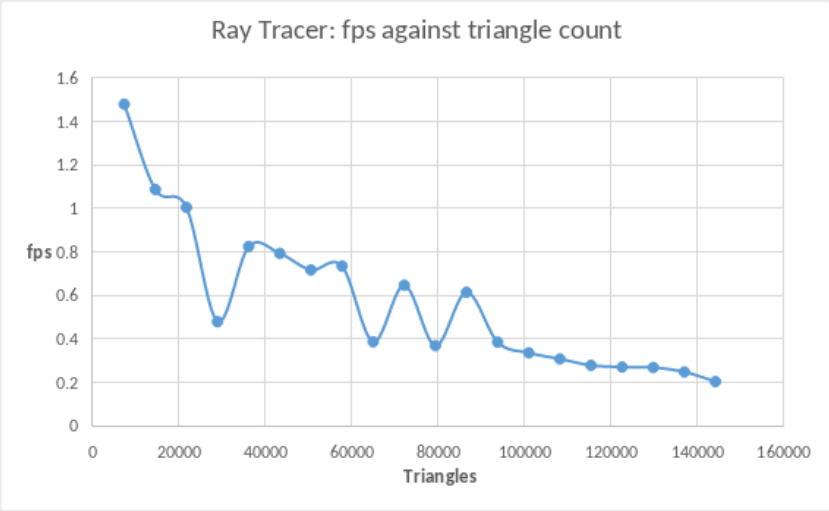


Figure 5.3: Shows the frames per second achieved by the rasterizer when rendering different amounts of triangles

Figure 5.3 shows the same results as in the rasterization pipeline: the time period (1/fps) for one frame to render has a logarithmic relationship to the number of triangles on the screen. This was predicted earlier, as the acceleration data structure used showed

a logarithmic traversal time. One thing to note is that this graph has a higher random variation between data points than the pixel count graph. This is probably due to random variation/fluctuations.

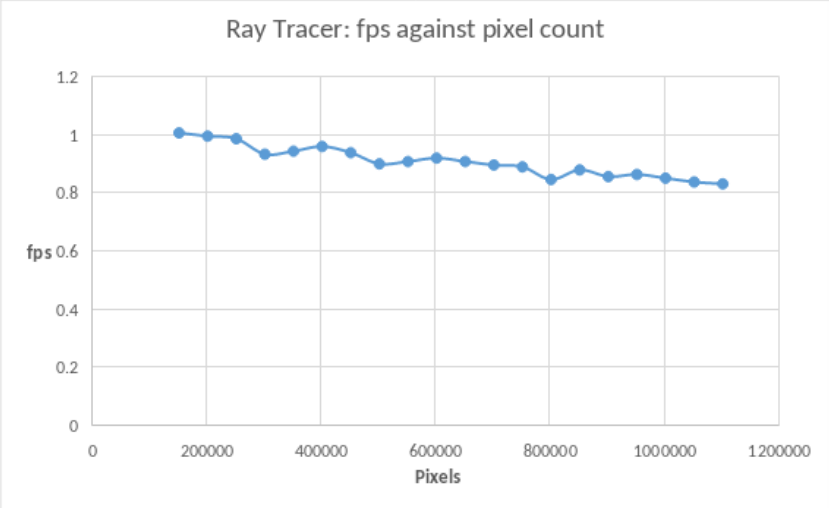


Figure 5.4: Shows the frames per second achieved by the rasterizer when rendering at different resolutions

Figure 5.4, like with the rasterization pipeline, shows that the fps has a slight downward trend as the resolution increases. This is because for each new pixel, there is a single new ray to handle, so the relationship is linear. But the rate of change is lower than expected, It was expected that the fps would reduce much faster. The unexpected results are because the scene used for testing could fit into a 100x100 screen, so each new ray only had to go through one ray-aabb intersection before being rejected, instead of the whole pipeline.

This shows that scene complexity, particularly how spread the scene is in the view frustum, largely affects the time it takes to render, otherwise there would be a greater slope.

5.3 Scene Complexity Requirements

For ***interactive real-time rendering***, the minimum frame rate is typically 30 frames per second (fps), for ***real-time rendering*** the minimum frame rate is typically 24 fps, or sometimes 12 fps for stylised renders, and for ***offline rendering***, there is no requirement for frame rates, all that matters is that it eventually gets rendered. In early Pixar films for example, some frames took days to render.

Tables 5.1 and 5.2 show how the fps of the render changes with the resolution and number of triangles, for both the rasterizer and ray tracer.

Rasterization Framerate Data:					
		Triangle count:			
		7202	14404	21606	144046
Resolution:	1920x1080	17.659	11.670	8.915	1.521
	1280x720	24.267	14.620	9.840	1.570
	848x480	30.511	15.823	10.855	1.582

Table 5.1: Two value data table showing rasterization framerates for varying resolution and triangle count.

Ray Tracing Framerate Data:					
		Triangle count:			
		7202	14404	21606	144046
Resolution:	1920x1080	0.914	0.769	0.710	0.027
	1280x720	1.189	0.949	0.866	0.055
	848x480	1.375	1.061	0.959	0.109

Table 5.2: Two value data table showing ray tracing framerates for varying resolution and triangle count.

The tables show that the rasterization pipeline can reach *interactive real-time rendering* frame rates (30fps) with 7202 triangles at 848x480 resolution; and *real-time rendering* frame rates (24fps) with 7202 triangles at 1280x720 resolution, or the stylised 12fps with 14404 triangles at 1920x1080 resolution.

The tables show that the ray tracing pipeline can't reach *interactive-real time* or even *real-time* frame rates for even simple scenes that have less than 10 thousand triangles.

Both the rasterization pipeline and ray tracing pipeline meet the requirement for *offline rendering* with 144046 triangles at 1920x1080 resolution, with the rasterization pipeline rendering a frame every 0.657 seconds, while the ray tracing pipeline renders a frame every 37.037 seconds.

5.4 Rasterization vs Ray Tracing renders

Figures 5.5 and 5.6 highlight the key differences in the quality of the images rendered with the rasterization pipeline and the ray tracing pipeline. While the rasterization pipeline is much faster than the ray tracing pipeline, it is not able to render images with nearly as high quality (left image).

The ray tracing image (right), however, uses complex ray tracing techniques and has access to complex shading algorithms, utilising the rendering equation to simulate the properties of light transportation.

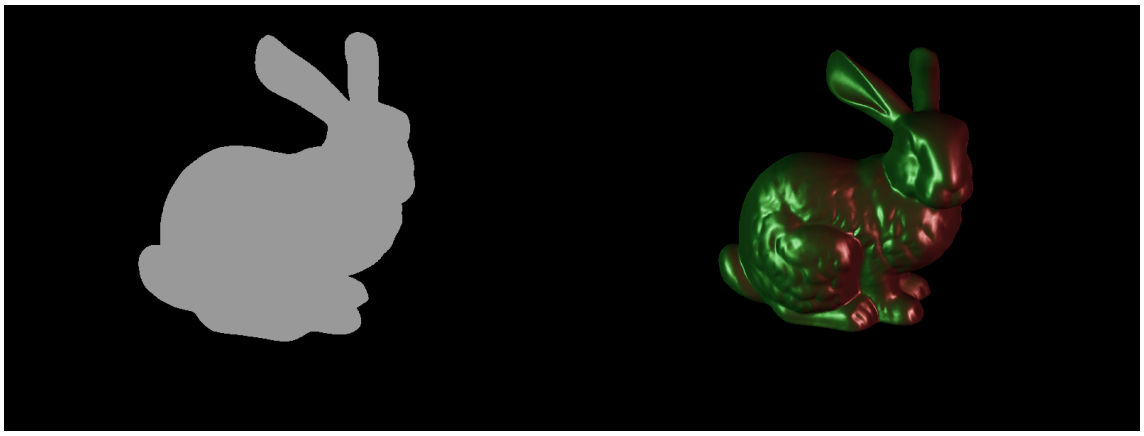


Figure 5.5: A bunny mesh with 86426 triangles rendered with the rasterization pipeline (left) and with the ray tracing pipeline (right)

The rasterization pipeline is not able to take into account light sources when rendering, so the images will look the same under all lighting conditions, and it is only able to use a crude form of shading shown in figure 5.6.

Figure 5.6: Triangle rendered with the rasterization pipeline

5.5 Final Renders

Figure 5.7 shows the final render, rendered with the ray tracing pipeline using offline rendering, that shows all three objects, shaded with the rendering equation and the Disney brdf[4] [3]. All objects are a greyish colour, and the colours you see are due to the 6 light sources in the scene:

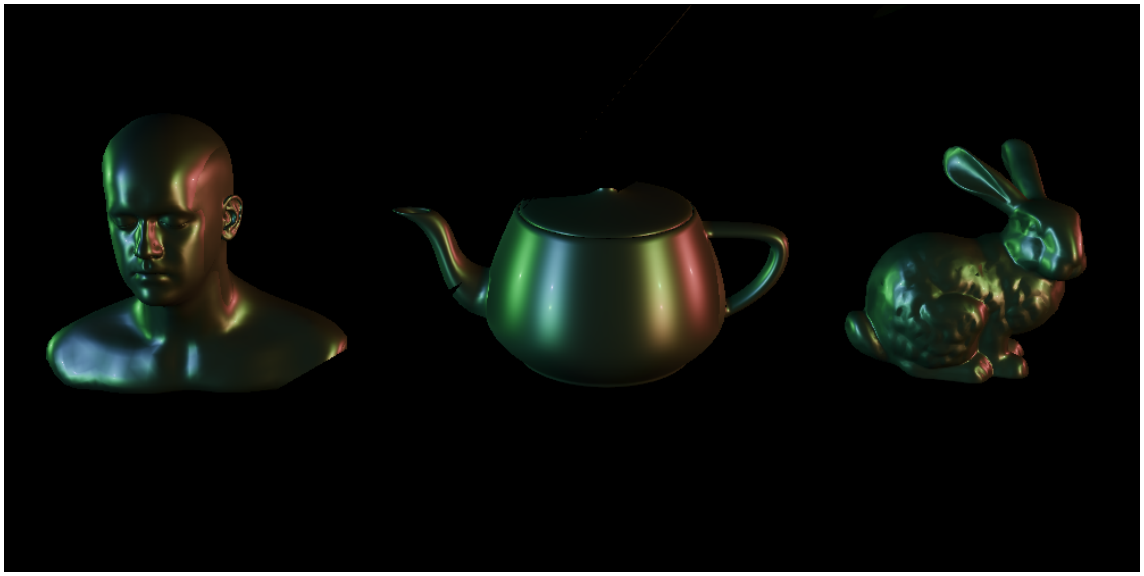


Figure 5.7: The final render

Chapter 6

Conclusions and Future Work

6.1 Review of aims

The aims of this project were to **develop a hybrid rasterized ray traced 3D graphics renderer on the CPU**, and to both qualitatively and quantitively compare the two rendering pipelines. **Qualitative comparison** in terms of the images rendered, and **quantitative measures** of their speed as the complexity of scenes varied with triangle count and resolution. It was also to determine if either of the two pipelines are fast enough to reach frame rates allowing for real time rendering, interactive real time rendering, and offline rendering, in order to determine if CPU rendering is feasible with modern processors.

A hybrid rasterized ray traced 3D graphics renderer was developed to run on the CPU, using a single thread. It includes dynamic scenes, allowing an application programmer to register meshes and lights to the scene, and change the position, scale, and rotation of meshes, lights and the camera as it is rendered. It includes a rasterization pipeline using the scanline algorithm and digital differential analyser algorithm; and a ray tracing pipeline using bounding volume hierarchies for ray-scene intersection; Moller-Trumbore ray-triangle intersection; barycentric vertex interpolation; and a shading interface implementing the rendering equation and the Disney brdf. Finally, it includes functionality for reading wavefront (.obj) mesh files, and for writing rendered scenes as a series of .ppm image files.

Qualitatively, the project showed that the ray tracing pipeline could render scenes at a much higher quality, taking into account the physics of light transportation, for various different materials. However, the rasterization pipeline, while much faster, could only render lower quality images.

Quantitatively, it was concluded that for both the rasterization and ray tracing pipelines, the rendering speeds follows a logarithmic relationship to the number of triangles in a scene, and a linear relationship to the number of pixels of the screen. It was also concluded that the rasterization pipeline could reach interactive real time rendering frame rates, real time rendering frame rates, and offline rendering frame rates with the scene complexity ranging from 7202 triangles at 848x480 resolution, to 144046 triangles at 1920x1080 resolution. It was determined that the ray tracing pipeline could not reach interactive real time rendering frame rates or real time rendering, but could reach offline rendering frame rates with 144046 triangles at 1920x1080 resolution.

6.2 Suggested Revisions to Design/Implementation

One revision is that in the current renderer, colours are stored on a per vertex basis rather than on a per triangle basis. At the time I thought that this would be good because it would mean the engine can handle rasterization shading by interpolating the colours at each vertex. But in hindsight, all this did was complicate things because for most complex meshes, a single triangle is smaller than a pixel, so you can't see the effects of the shading anyway.

Moreover, imagine that an artist wanted to shade a whole mesh such that one side was one colour, and the other side was another colour, and in the middle, the colour slowly changed from one to the other. In this case, shading within triangles doesn't matter, what actually matters is that the colours of the vertices themselves are shaded properly, and this is done by the artist in modelling software, not by the renderer. Hence the only real use case for per vertex shading is for meshes with a small number of triangles, but the speed gained from doing this is counteracted by the speed loss in shading the triangles, and the added complexity of the renderer.

Another revision is that currently, each mesh stores UV coordinates. The reasoning for this is that some bxdf evaluation functions require the orthogonal basis vectors for the plane of the triangle being shaded, and the calculation for this includes the UV coordinates. But this is only necessary if the bxdf shades differently depending on the orientation of the triangle - but this isn't true for most bxdfs. It is only true for specific bxdfs such as a metal plate bxdf that has a horizontal scratch in it. For this bxdf, the orientation does matter because this scratch will affect how light reflects from it. This renderer doesn't implement such brdfs, so arbitrary basis vectors can be selected, without needing the UV coordinates for the calculation.

The final revision is in the logic for reading in mesh files. Currently, it is setup to load in wavefront (.obj) files, but these files don't include colour data. It would be better to support a different file type such as the graphics library transmission format (.gltf), that includes colours.

6.3 Future Work

Future work could be spent looking into how CPU concurrency increases the speed of both the rasterization pipeline and ray tracing pipeline, as well as the effect that different acceleration data structures, such as different versions of kd-trees, and Octrees, with different splitting heuristics, will have on the ray tracing speed.

Work could be spent looking into how an engine that renders some objects with rasterization and some with ray tracing would look like. Moreover, work could be spent looking into the more computationally expensive ray tracing algorithms such as recursive ray tracing, and path tracing. Additional work could be spent looking into a different shading interface, where cameras, surfaces, lights, and the environment have different programmable shaders with different interfaces.

6.4 Lessons Learned

I learned 3D graphics from the ground up, including: how meshes are stored; the rasterization and ray tracing pipelines; as well as physically based rendering principles, the rendering equation and bidirectional distribution functions. I also learned the mathematics of 3D rigid body transformations, affine transformations, projections and camera transformations.

I learnt the limitations of single threaded CPU rendering, and the use cases of rasterization and ray tracing running on a single CPU thread.

I learned the advantages and disadvantages of object oriented programming, and imperative programming, where and how they should be used in code, and how to combine them to make multi-paradigm programs.

6.5 Final Remarks

Overall, I managed to make a renderer that I am quite proud of, and which achieves all that I wanted it to for this project. I feel like I really understand how graphics works from a low level, and now know enough to know what I don't know. I am excited to dive deeper into the subject, learn more about 3D rendering in the context of physically based rendering, and write more, more powerful renderers utilising rendering APIs, the GPU, and techniques such as bidirectional path tracing and photon mapping.

Appendix A

Project Proposal

Win3D Project Proposal

Background and Introduction

Computer graphics are becoming more and more prevalent in the modern age, from the graphics you are looking at to read this paper to animation and CGI in films and TV, to graphics used in video games. 3D graphics in particular, are becoming more prevalent with the rise of 3D animation studios, and game studios. 3D graphics is a subset of computer graphics, where 3 dimensional objects, stored in a 3D vector space, have to be rendered onto a screen of just 2 dimensions. This comes with a host of new problems over traditional 2D graphics, and there have been lots of different solutions to these problems. There are different ways to store coordinates (cartesian or polar), different ways to handle rotation (euler matrices or quaternions), and different pipelines that transform object data into pixels on our screen (forward rendering vs deferred rendering).[1]

With this has come the rise of the GPU - and the more modern GPGPU - a separate processor to the CPU that is specifically designed to handle massively parallel operations, such as graphics processing operations.[2] Graphics APIs have also been developed to make easier the lives of programmers who are trying to render graphics to the screen. Graphics APIs and the GPU, unfortunately, abstract away a lot of what is necessary to render graphics to the screen, such as the graphics rendering pipeline. Moreover, the graphics rendering pipeline, includes stages that offer differing levels of customisability/programmability, dependent on the specific vendor.[3] Some stages, such as the rasterization stage, offer no ability for it to be configured or programmed in any way; it has a set, fixed-function. Other stages offer varying levels of configurability, but you cannot directly program them. Therefore, my goal is to write my own graphics rendering system from scratch, on the CPU, without the use of the GPU or any graphics API.

Within 3D graphics rendering, there is an area called physically based rendering, where graphics rendering systems are designed to be accurate to physics and the real world. Physically based rendering systems must take into account physical laws such as the conservation of energy, etc. physically based rendering is often used for photo realistic rendering, which is what I will be using it for, but it is still capable of creating stylised effects. There is also the question of offline vs real-time rendering, where graphics are either rendered in real-time, as they are being shown to the customer; or beforehand, such that their quality can be increased, and shown to the customer later. This project will mainly focus on developing a physically-based, offline rendering system.[4]

The Proposed Project

The main goal of this project is to develop a graphics API that implements the graphics rendering pipeline, and physically-based offline rendering techniques, such that users can develop and render high quality 3D graphics and scenes. Programmers using the API will be able to: create a “GraphicsPipeline” object composed of the different stages of the graphics pipeline; set different pipeline parameters such as the projection used; use it to pass imported objects through the pipeline; and inherit from it if they want to edit the pipeline in any way. The API should also have abstractions for high level application layer concepts like different objects (squares, spheres, illumination objects, viewports, etc) and shading techniques like raytracing, volumetric effects, caustic effects, etc. So in essence, it is an implementation of the graphics rendering pipeline with an API abstracting away each of its stages, including application layer abstractions, with the main purpose being for offline, physically based/photorealistic rendering.

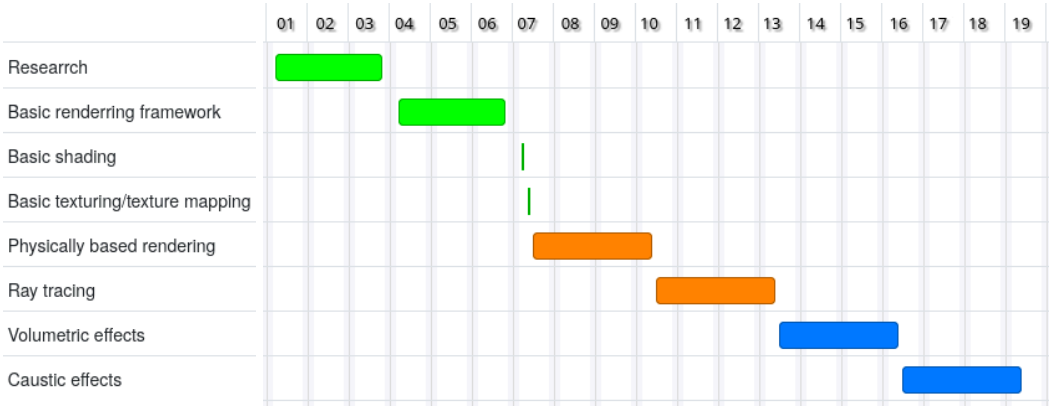
Project plan for implementation

Programme of work

- Research (3 weeks)
    - Research into the topic of 3D graphics, what will need to be implemented, and related topics.
- Basic rendering framework (3 weeks)
    - Includes the basic graphics rendering pipeline: the application phase; the geometry processing stage; the rasterization stage; and the pixel processing stage.
    - The ability to import objects from blender.
    - The ability to pass objects through the whole pipeline via a draw call.
- Basic shading (1 day)
    - Phong shading
- Basic texturing/texture mapping (1 day)
    - Can import and map textures to objects in the pixel processing stage of the graphics rendering pipeline
- Physically based rendering (3 weeks)
    - Implementations of various Bidirectional reflectance distribution functions
    - The ability to do physically based texturing
- Ray tracing (3 weeks)
    - Basic ray tracing
- Volumetric effects (3 weeks)
    - Effects for smoke, fire, clouds, etc
- Caustic effects (3 weeks)
    - Effects for glass, water, etc

Gantt chart

Green: Milestone 1; Orange: Milestone 2; Blue: Milestone 3.



References

[1] GOMES, J. (2017) *DESIGN AND IMPLEMENTATION OF 3D GRAPHICS SYSTEMS*. First edition. Place of publication not identified: CRC Press.

[2] Hwu, W. (2012) *GPU computing gems*. Jade ed. Boston, Mass: Morgan Kaufmann.

[3] Akenine-Moller, T. *et al.* (2018) *Real-time rendering*. Fourth edition. Boca Raton: CRC Press.

[4] Matt Pharr, Wenzel Jakob, and Greg Humphreys. (2016) Physically Based Rendering: From Theory to Implementation (3rd ed.). Morgan Kaufmann Publishers Inc.

Bibliography

[1] Lehtinen Bernhard Kerbl, Jakko. Rendering: Spatial acceleration structures, 2020.

[2] Jakub Boksansky. Crash course in brdff implementation. `https://boksajak.github.io/blog/BRDF`, February 2021.

[3] Brent Burley. Physically-based shading at disney. In *Physically-Based Shading at Disney*, 2012.

[4] Brent Burley. Walt disney animaiton studios: Disney brdff https://github.com/wdas/brdff/commits/main/src/brdfs/disney.brdff, April 2017.

[5] Robert L. Cook, Thomas Porter, and Loren Carpenter. Distributed ray tracing. *SIGGRAPH Comput. Graph.*, 18(3):137–145, January 1984.

[6] Joey de Vries. Learn opengl: Learn modern opengl graphics programming in a step-by-step ffashion, 2020.

[7] Brendan Galea. The math behind (most) 3d games - perspective projection. Youtube, June 2021.

[8] JBikker. How to build a bvh - part 1: Basics, April 2022.

[9] James T. Kajiya. The rendering equation. *SIGGRAPH Comput. Graph.*, 20(4):143–150, August 1986.

[10] Wenzel Jakob Matt Pharr and Greg Humphreys. *Physically Based Rendering, ourth edition: From Theory to Implementation, 7.3 Bounding Volume Hierarchies.* MIT Press, 2023.

[11] Tomas Möller and Ben Trumbore. Fast, minimum storage ray/triangle intersection. In *ACM SIGGRAPH 2005 Courses*, SIGGRAPH '05, page 7–es, New York, NY, USA, 2005. Association ffor Computing Machinery.

[12] Steve Hollasch Peter Shirley, Trevor David Black. Ray tracing in one weekend, April 2025. `https://raytracing.github.io/books/RayTracingInOneWeekend.html`.

[13] Steve Hollasch Peter Shirley, Trevor David Black. Ray tracing: The next week, April 2025. `https://raytracing.github.io/books/RayTracingTheNextWeek.html`.

[14] Jean-Colas Prunier. Barycentric coordinates, 2009.

[15] Niklas Sanden. Bounding volume hierarchy construction. Technical report, Lund University, 2022.

[16] Naty Hoffman Angelo Pesce Michal Iwanicki Tomas Akenine-Moller, Eric Haines and Sebastien Hillaire. *Real-Time Rendering, Fourth Edition, 11-55.* A K Peters/CRC Press, 2018.

[17] Naty Hoffman Angelo Pesce Michal Iwanicki Tomas Akenine-Moller, Eric Haines and Sebastien Hillaire. *Real-Time Rendering, Fourth Edition, 58-76.* A K Peters/CRC Press, 2018.

[18] Bal´azs T´oth. Speed optimized recursive ray-tracer with kd-tree and sse vector mathematics. Technical report, Citeseer, n.d.

[19] Ingo Wald. *Realtime Ray Tracing and Interactive Global Illumination.* PhD thesis, Computer Graphics Group, Saarland University, 2004.

[20] Chris Wynn. An introduction to brdf-based lighting. *NVIDIA Corporation.*